

Implementation of a Spectrogram-Based Ensemble Deep Learning Framework for Deepfake Audio Detection

Project Report Submitted

by

MUHAMMED MUZAMMIL A

MG24C7115001

Under the Guidance of

Dr. Aljinu Khadar K V

Assistant Professor

Data Science and Analytics

Department of Computer Science

University of Calicut

&

Dr. SHARON SUSAN JACOB

School of Data Analytics

Mahatma Gandhi University – Kottayam

in partial fulfilment for the award of the degree of

MASTER OF SCIENCE

in

DATA SCIENCE AND ANALYTICS



SCHOOL OF DATA ANALYTICS

MAHATMA GANDHI UNIVERSITY

KOTTAYAM

June, 2026



UNIVERSITY OF CALICUT

School of Mathematical and Computational Sciences

Department of Computer Science

CERTIFICATE

This is to certify that **Mr. Muhammed Muzammil A**, Register Number **MG24C7115001**, a student of **M.Sc. Data Science and Analytics**, School of Data Analytics, Mahatma Gandhi University, Kottayam, has successfully completed his project titled: **“Implementation of a Spectrogram-Based Ensemble Deep Learning Framework for Deepfake Audio Detection”** under my supervision and guidance during the period from **02 February 2026 to 31 May 2026**.

The project was carried out as part of the academic requirements of the M.Sc. Data Science and Analytics programme. During the course of the project, he demonstrated sincere effort, commitment, and technical competence in completing the work. The project has been completed to my satisfaction.

University of Calicut

02/06/2026

Aljinu Khadar K V
02/06/2026

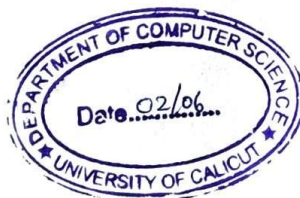
Dr. Aljinu Khadar K V

Assistant Professor (Data Science & Analytics)

Department of Computer Science

University of Calicut

Kerala, India



CERTIFICATE

This is to certify that the dissertation entitled “**Implementation of a Spectrogram-Based Ensemble Deep Learning Framework for Deepfake Audio Detection**” is an original and authentic record of study carried out by **MUHAMMED MUZAMMIL A** under my guidance and supervision, in partial fulfilment of the requirement for the award of the degree of Master of Science, during the academic year 2024-2026 and that it has not been previously submitted to the award of any degree, diploma, fellowship or any other similar title or recognition.

Dr. Sharon Susan Jacob

Project Guide

Dr. K. K. Jose

Director

School of Data Analytics

DECLARATION

I hereby declare that the project entitled “**Implementation of a Spectrogram-Based Ensemble Deep Learning Framework for Deepfake Audio Detection**” submitted by me in partial fulfilment of the requirements for the award of Degree of Master of Science in Data Science and Analytics, is a Bonafide work done by me under the guidance of **Dr Aljinu Khadar K V**, Assistant Professor, Data Science and Analytics, Department of computer science University of Calicut, Thenhipalam, Malappuram and **Dr. Sharon Susan Jacob**, Assistant Professor, School of Data Analytics, Mahatma Gandhi University, Kottayam.

To the best of my knowledge, it is an original work, both in terms of research content and narrative, and has not been submitted or accepted elsewhere, in part or in full, for the award of any degree, diploma, fellowship, or similar title of any university or institution.

MUHAMMED MUZAMMIL A

School of Data Analytics

Mahatma Gandhi University, Kottayam

ACKNOWLEDGEMENT

At the outset, I thank God Almighty for making the endeavour a success. The success and final outcome of this project required a lot of guidance and assistance from many people and I am extremely fortunate to have this all along the completion of my project work. Whatever I have done is only due to such guidance and assistance and I would not forget to thank them. I would like to express my profound gratitude to my guide **Dr Aljinu Khadar K V**, Assistant Professor, Data Science and Analytics, Department of computer science University of Calicut, Thenhipalam, Malappuram, for his valuable time and guidance that I have received throughout the period of this work. I express my deep sense of gratitude to **Dr. K.K Jose**, Honourable Director, School of Data Analytics, for his support and guidance. I owe my profound gratitude to my internal project guide **Dr. Sharon Susan Jacob**, Assistant Professor, School of Data Analytics, who took keen interest on my project work and guided me all along, by providing all the necessary information for developing a good project. I would like to thank the entire faculty of the School of Data Analytics, Mahatma Gandhi University, for their kind support at every stage of the project. At last, but not the least I wish to thank my friends and family who have helped me directly and indirectly in the successful completion of this project work.

MUHAMMED MUZAMMIL A

ABSTRACT

Voice synthesis and voice conversion technologies have advanced rapidly in recent years. It is now increasingly easy to generate realistic fake speech. This has become a serious concern for automatic speaker verification (ASV) systems, which are widely used in security applications. Detecting such synthesised or converted speech, commonly referred to as deepfake audio, is a challenging problem that has attracted growing research attention.

This project implements a Spectrogram-Based Ensemble Deep Learning Framework for Deepfake Audio Detection and evaluates it on the ASVspoof2019 Logical Access (LA) dataset. The central idea is a multi-branch architecture that takes three different time-frequency representations of audio: Short-Time Fourier Transform (STFT), Mel Spectrograms, Constant-Q Transform (CQT) Spectrograms, and Continuous Wavelet Transform (CWT) Spectrograms. Each is processed through a separate parallel branch before feature combination for classification. Each spectrogram is further enriched with delta and delta-delta dynamic features. This forms a $(3 \times 64 \times 64)$ input tensor per branch. This design allows the model to capture a wider range of acoustic artefacts than a single spectrogram type could.

Three models are built and evaluated under this framework. The first is a Multi-Branch Custom CNN, where three parallel convolutional branches process the three spectrogram types, and their features are concatenated before classification. The second is a Multi-Branch ResNet-50, which follows the same branch structure but uses ImageNet-pretrained ResNet-50 backbones for each branch. The third is a Whisper-based MLP, which uses the encoder of OpenAI's Whisper speech model, pretrained on 680,000 hours of multilingual audio, to extract 512-dimensional embeddings that are then classified through a lightweight MLP. These three models are combined into an ensemble, and seven different fusion weight strategies are compared.

Among the individual models, the Whisper MLP performed best with an accuracy of 96.74%, AUC of 0.9869, and EER of 5.60%. The Multi-Branch CNN achieved an AUC of 0.9289 and EER of 14.69%, while the ResNet-50 achieved an AUC of 0.8926 and EER of 18.36%. The equal-weight ensemble of all three models (CNN + ResNet-50 + Whisper) achieved EER of 6.91% and AUC of 0.9836. The best ensemble configuration, using a weighted fusion of Whisper (0.8) and CNN (0.2), achieved the lowest EER of 4.52%, showing that combining these two complementary models gives better results than either alone.

CONTENTS

1. INTRODUCTION.....	9
1.1 Background and Motivation	9
1.2 Problem Statement	10
1.3 Objectives	10
1.4 Organisation of the Report	11
2. LITERATURE REVIEW	12
3. METHODOLOGY	15
3.1 Overview	16
3.2 Dataset	16
3.3 Preprocessing Pipeline	16
3.4 Multi-Branch Custom CNN	18
3.5 Multi-Branch ResNet-50	19
3.6 Whisper-Based MLP.....	19
3.7 Ensemble Fusion	20
3.8 Overfitting Mitigation Strategies	21
4. EXPERIMENTAL RESULTS AND ANALYSIS.....	24
4.1 Evaluation Metrics	24
4.2 Overall Model Performance.....	25
4.3 Model Accuracy Comparison.....	32
4.4 AUC-ROC Comparison	33
4.5 EER Comparison.....	34
4.6 ROC Curves.....	35
4.7 Training Dynamics	36

4.8 Ensemble Confusion Matrix	37
4.9 Ensemble Fusion Strategy Comparison	38
4.10 Summary	39
5. CONCLUSION AND FUTURE WORK	40
5.1 Conclusion	40
5.2 Future Work	41
References	42
Appendix – Python Source Code	44

CHAPTER 1

INTRODUCTION

Voice-based authentication has become a common feature in many everyday applications from mobile banking to smart home devices. The reliability of these systems depends on their ability to tell apart real speech from fake. But with modern text-to-speech and voice conversion tools becoming more accessible, producing convincing synthetic speech no longer requires expert knowledge. This makes it easier than ever to attempt spoofing attacks against speaker verification systems, and it raises a genuine need for robust detection methods.

Deepfake audio detection, also called anti-spoofing, is the task of automatically deciding whether a given speech sample is genuine or synthetically produced. Over the past decade, several benchmark challenges most notably the ASVspoof series have been organised to drive progress in this area. The 2019 edition of ASVspoof introduced the Logical Access (LA) scenario, which covers a range of text-to-speech (TTS) and voice conversion (VC) systems as spoofing sources. It has since become the standard benchmark for evaluating anti-spoofing countermeasures.

Most existing systems rely on a single type of audio feature typically a Mel spectrogram to train a classifier. While these approaches have shown good results, they may miss artefacts that are only visible in other types of time-frequency representations. Different spectrogram types capture different characteristics of speech: Mel spectrograms capture perceptually weighted energy; CQT spectrograms offer better frequency resolution at low frequencies; and wavelet spectrograms provide multi-resolution analysis suited to capturing transient events. Using all three together, through a multi-branch architecture, is the core idea behind this project.

This work proposes and evaluates a spectrogram-based ensemble deep learning framework for deepfake audio detection. Three models are trained a Multi-Branch CNN, a Multi-Branch ResNet-50, and a Whisper-based MLP and their predictions are combined through a soft-probability ensemble. The framework is evaluated on the ASVspoof 2019 LA benchmark using accuracy, AUC-ROC, and Equal Error Rate (EER).

1.1 Background and Motivation

Automatic speaker verification systems rely on the assumption that each person's voice has unique characteristics that cannot easily be replicated. This assumption breaks down when high-quality speech synthesis is available. Neural TTS models such as WaveNet, Tacotron-2, and VITS can now produce speech that sounds natural even to careful listeners, making automated detection the only practical line of defence.

The motivation for a multi-branch, multi-representation approach comes from a simple observation: no single spectrogram type captures everything. STFT Mel spectrograms are perceptually motivated but may miss fine frequency details. CQT spectrograms are better at resolving low-frequency pitch structures. Wavelet spectrograms are more sensitive to short-duration transient events. Combining all three within a single model, rather than choosing one arbitrarily, should give a more complete picture of the audio signal.

Additionally, the availability of large pretrained speech models like Whisper creates a new opportunity. These models have seen enormous amounts of speech data and their internal representations carry rich information about acoustic structure. Using Whisper as a feature extractor without fine-tuning it is a simple but powerful approach that this project explores alongside the spectrogram-based branches.

1.2 Problem Statement

Most anti-spoofing systems in the literature use a single feature type, which limits their ability to detect the full range of synthesis artefacts. Ensemble methods for deepfake audio detection are relatively underexplored, and the question of how to best combine heterogeneous models particularly those mixing CNN-based spectrogram models with large pretrained speech encoders has not been carefully studied. This project addresses these gaps directly.

1.3 Objectives

1. To design a multi-branch architecture that processes STFT, CQT, and CWT spectrograms through parallel CNN branches for deepfake audio detection.
2. To train and evaluate three model variants Multi-Branch CNN, Multi-Branch ResNet-50, and Whisper MLP on the ASVspoof 2019 LA dataset.
3. To combine the three models into an ensemble and compare seven different fusion weight strategies.

4. To evaluate all models using Accuracy, AUC-ROC, and EER.
5. To test the trained ensemble on unseen audio samples to assess practical usability.

1.4 Organisation of the Report

Chapter 2 reviews relevant prior work on deepfake audio detection, spectrogram representations, and ensemble methods. Chapter 3 describes the dataset, preprocessing steps, model architectures, and training setup. Chapter 4 presents and discusses the experimental results. Chapter 5 reflects on the findings and their implications. Chapter 6 concludes the report and suggests directions for future work.

CHAPTER 2

LITERATURE REVIEW

The detection of synthesised speech has become an increasingly active area of research, driven largely by the ASVspoof challenge series and the rapid improvement of neural speech synthesis systems. This chapter reviews seven closely related works that cover the main approaches relevant to this project spectrogram-based detection, the use of pretrained speech models, multi-branch architectures, and ResNet-based systems. The gaps that this work addresses are discussed at the end.

Spectrogram-Based Ensemble Systems

The most directly related prior work is that of Tran et al. [1], who proposed a comprehensive spectrogram-based ensemble framework for deepfake audio detection on ASVspoof 2019 LA. Their system extracts multiple spectrogram types using STFT, CQT, and Wavelet transforms with various filter banks and trains three types of models on them: CNN/RNN baselines, transfer learning models (ResNet, MobileNet, EfficientNet, DenseNet), and audio embedding models (Whisper, SpeechBrain, Pyannote) whose outputs are passed through an MLP classifier. Each spectrogram type is treated as a separate input to independent models, and their best ensemble achieves an EER of 0.03 on the ASVspoof 2019 LA dataset.

This work provides strong motivation for the approach taken in this project. The key difference is that rather than training separate models for each spectrogram and then ensembling them externally, this project combines all three spectrogram types within a single multi-branch model, so that features from different representations are learned jointly before the classification decision is made.

Whisper for Deepfake Detection

Kawa et al. [2] studied the use of Whisper features as front-end representations for deepfake audio detection. They tested three detection models LCNN, SpecRNet, and MesoNet — trained on Whisper-based features from the ASVspoof 2021 DF dataset, and found that Whisper features consistently improved detection performance across all three models, reducing EER by 21% compared to conventional front-ends. The results generalised well to the In-The-Wild dataset, showing that Whisper representations transfer effectively to real-world conditions.

Qian et al. [3] took a similar direction, combining Whisper's encoder with the AASIST anti-spoofing architecture. Their work showed that using Whisper encoder embeddings as input features — rather than conventional spectrograms — improved AASIST's detection capability on the ASVspoof benchmark. Together, these two works directly motivated the inclusion of a Whisper-based MLP branch in this project. Unlike these studies, which use Whisper as a replacement for conventional features, this project combines the Whisper branch with spectrogram-based CNN models in an ensemble, benefiting from both types of representation simultaneously.

Multi-Branch and Multi-Resolution Architectures Wang et al. [4] proposed a multi-resolution feature map approach for anti-spoofing CNNs. Their idea was to stack multiple spectrograms extracted using different window lengths as separate input channels to a CNN, improving both frequency and time resolution compared to a single-resolution input. Experiments on ASVspoof 2019 confirmed that this multi-resolution approach consistently outperforms score fusion of separately trained models, with only a small additional computational cost.

This study is closely related to the multi-branch design used here. The main difference is that Wang et al. vary the window length of the same STFT-based transform to get multiple resolutions, whereas this project combines three fundamentally different transforms STFT, CQT, and CWT each of which captures a structurally different aspect of the audio signal.

Cheng et al. [6] designed a multi-branch ResNet architecture called ResNeWt for replay attack detection in ASVspoof 2019. Their multi-branch building block was shown to improve generalisation and reduce feature redundancy compared to standard ResNet, by allowing different branches to focus on different discriminative features. This work provided a direct architectural precedent for the Multi-Branch ResNet-50 used in this project.

ResNet-Based Anti-Spoofing Systems

Tak et al. [5] investigated the Res2Net architecture for both replay and synthetic speech detection on ASVspoof 2019. Their results showed that Res2Net, which incorporates multi-scale feature extraction within each residual block, significantly outperformed standard ResNet-34 and ResNet-50 for both physical and logical access scenarios. This confirmed that residual architectures with multi-scale capability are particularly effective for anti-spoofing, supporting the use of ResNet-50 as a backbone in this project.

Gomez-Alanis et al. [7] applied deep residual networks to ASVspoof 2019, treating spectrograms as two-dimensional images and leveraging standard CNN architectures from image classification. Their work demonstrated that pretrained convolutional architectures can be effectively adapted to the anti-spoofing task through transfer learning, directly informing the use of ImageNet pretrained ResNet-50 weights in this project.

Gaps Addressed

Based on the works reviewed above, four gaps are identified and addressed in this project.

Single spectrogram dependency. Most systems use one spectrogram type. This project uses three STFT, CQT, and CWT processed through parallel branches within a single model, allowing joint feature learning across representations.

Limited Whisper-CNN hybrid ensembles. Kawa et al. [2] and Qian et al. [3] use Whisper within single-model frameworks. This project combines a Whisper MLP with multi-branch spectrogram CNNs in an ensemble and evaluates multiple fusion weight strategies between these different model types.

Unsystematic fusion strategy evaluation. Most prior ensemble systems use equal-weight averaging. This project systematically compares seven different fusion strategies, providing concrete guidance on how to weight heterogeneous model combinations.

CHAPTER 3

METHODOLOGY

3.1 Overview

The pipeline implemented in this project has three main stages. First, raw audio files from the ASVspoof 2019 LA dataset are loaded, segmented into 2-second windows, and converted into three types of spectrogram tensors. Second, three deep learning models are trained on these representations: a Multi-Branch CNN, a Multi-Branch ResNet-50, and a Whisper-based MLP. Third, the predictions from these three models are combined through a soft-probability ensemble, and seven different fusion weight configurations are compared to find the best-performing strategy.

3.2 Dataset

3.2.1 ASVspoof 2019 Logical Access Dataset

The ASVspoof 2019 Logical Access (LA) dataset is used for all experiments. It covers a range of TTS and voice conversion systems as spoofing sources, and all audio files are single-channel FLAC recordings at 16 kHz, drawn from the VCTK speaker pool. Ground-truth labels are parsed from the official protocol files: bonafide speech is assigned label 1 and spoofed speech is assigned label 0.

The dataset has three splits: training, development, and evaluation. Spoofing systems A01–A06 appear in the training and development sets, while thirteen additional unseen systems (A07–A19) are reserved for evaluation only. This mismatch between training and evaluation conditions is intentional; it tests how well a system generalises to spoofing methods it has never seen before, which is the most important challenge in practical anti-spoofing.

3.2.2 Dataset Statistics

Table 3.1: ASVspoof 2019 LA Dataset Folder Structure

Folder	Contents
ASVspoof2019_LA_train	Training split audio files (.flac)
ASVspoof2019_LA_dev	Development split audio files (.flac)
ASVspoof2019_LA_eval	Evaluation split audio files (.flac)
ASVspoof2019_LA_cm_protocols	Countermeasure protocol label files
ASVspoof2019_LA_asv_protocols	ASV system protocol files
ASVspoof2019_LA_asv_scores	Pre-computed ASV system scores

Table 3.2: ASVspoof 2019 LA Partition Statistics

Partition	Bonafide	Spoofed	Total	Bonafide:Spoof
Train	2,580	22,800	25,380	1 : 8.8
Development	2,548	22,296	24,844	1 : 8.7
Evaluation	7,355	63,882	71,237	1 : 8.7
Total	12,483	108,978	121,461	1 : 8.7

The class imbalance is consistent across all three splits at roughly 1:9. This is an important consideration when interpreting accuracy-based results, since a model that always predicts spoof would already achieve around 89% accuracy. This is why EER and AUC-ROC are used as the primary performance measures. To keep the evaluation computationally manageable, a random sample of 10,000 files was drawn from the full evaluation set using a fixed seed (seed = 42).

3.3 Preprocessing Pipeline

3.3.1 Audio Segmentation

All audio is loaded using torchaudio and resampled to 16,000 Hz if needed. Each recording is then divided into fixed 2-second windows of 32,000 samples. If the last segment of a file is shorter than 2 seconds, it is zero-padded on the right. Every segment inherits the binary label of the parent file.

3.3.2 Spectrogram Feature Extraction

Three spectrogram types are computed for each 2-second segment, each producing a 64×64 matrix.

STFT Mel Spectrogram: A Mel-scale filter bank with 64 filters is applied to the STFT output ($n_fft = 512$, $hop_length = 128$). The result is converted to decibels using `librosa.power_to_db`. This gives a perceptually weighted view of the audio's frequency content.

CQT Spectrogram: Computed using `librosa.cqt` with $n_bins = 64$ and $hop_length = 128$. The magnitude is converted to decibels. The CQT uses logarithmically spaced frequency bins, which gives better resolution at low frequencies useful for detecting pitch-related synthesis artefacts.

CWT Spectrogram: Computed using the Mexican Hat wavelet (`mexh`) via `PyWavelets` across 64 scales, applied to the first 2,048 samples of each segment. The wavelet transform provides multi-resolution analysis, making it sensitive to short-duration transient events that may be present in synthesised speech.

3.3.3 Delta Features and Tensor Construction

For each spectrogram, first-order (delta) and second-order (delta-delta) temporal derivatives are computed using `librosa.feature.delta`. These capture how the spectral content changes over time, which is particularly useful for detecting unnatural transitions in synthetic speech. The base spectrogram, delta, and delta-delta are each normalised to $[0, 1]$ using min-max normalisation, then stacked along the channel dimension to form a $(3, 64, 64)$ tensor similar to an RGB image.

Each audio segment therefore produces three tensors one for STFT, one for CQT, one for CWT saved to disk as .pt files with the naming convention:

`{file_id}_seg{N}_label{Y}_{type}.pt`

where N is the segment index, Y is the binary label, and `type` is one of `stft`, `cqt`, or `wav`.

3.4 Multi-Branch Custom CNN

3.4.1 Architecture

The Multi-Branch CNN has three parallel convolutional branches one for each spectrogram type. All branches have the same structure. Each branch takes a (3, 64, 64) tensor and passes it through three convolutional blocks:

- Block 1: Conv2d(3→32) → BatchNorm → ReLU → MaxPool(2×2) → Dropout2d(0.2) → output: 32×32
- Block 2: Conv2d(32→64) → BatchNorm → ReLU → MaxPool(2×2) → Dropout2d(0.3) → output: 16×16
- Block 3: Conv2d(64→128) → BatchNorm → ReLU → MaxPool(2×2) → Dropout2d(0.4) → Flatten → 8,192-dim vector

The outputs of the three branches are concatenated into a single 24,576-dim vector. This goes through the classifier head:

Linear(24,576→512) → ReLU → Dropout(0.6) → Linear(512→256) → ReLU → Dropout(0.5) → Linear(256→1)

The increasing dropout rates across layers (0.2 → 0.3 → 0.4 → 0.6 → 0.5) are a deliberate regularisation strategy to prevent overfitting given the limited size of the minority bonafide class.

3.4.2 Training

- Loss: BCEWithLogitsLoss
- Optimiser: Adam (lr = 1×10^{-3} , weight decay = 1×10^{-4})
- Scheduler: StepLR (step = 5, gamma = 0.5)
- Epochs: 20, batch size: 32, early stopping patience: 5
- Best model saved to: best_cnn.pth

3.5 Multi-Branch ResNet-50

3.5.1 Architecture

Three ResNet-50 backbones are used one per spectrogram type each initialised with ImageNet pretrained weights (IMAGENET1K_V2). The final fully connected layer of each backbone is replaced with `nn.Identity()`, so each branch outputs a 2,048-dim feature vector. The three vectors are concatenated into a 6,144-dim representation and passed through:

$\text{Linear}(6,144 \rightarrow 512) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.6) \rightarrow \text{Linear}(512 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(256 \rightarrow 1)$

All three backbones and the classifier head are fine-tuned end-to-end.

3.5.2 Training

- Loss: BCEWithLogitsLoss
- Optimiser: Adam ($\text{lr} = 1 \times 10^{-4}$, weight decay = 1×10^{-4})
- Scheduler: StepLR (step = 5, gamma = 0.5)
- Epochs: 10, batch size: 32, early stopping patience: 3
- Best model saved to: best_resnet.pth

The lower learning rate compared to the CNN is intentional it preserves the pretrained ImageNet features while still allowing the model to adapt to the deepfake detection task.

3.6 Whisper-Based MLP

3.6.1 Feature Extraction

The encoder of OpenAI's **Whisper-base** model is used as a frozen feature extractor. For each audio segment, the Whisper encoder produces a sequence of **512-dimensional** frame-level representations. Mean pooling across the time dimension gives a single 512-dim embedding per segment. The encoder weights are kept frozen throughout no gradients flow into Whisper which keeps training fast and prevents catastrophic forgetting.

Since encoding is computationally expensive, all Whisper embeddings are pre-extracted offline and saved as .npy files before training begins. The MLP classifier is then trained on these saved embeddings:

Linear(512→256) → ReLU → Dropout(0.4) → Linear(256→128) → ReLU → Dropout(0.3) → Linear(128→1)

3.6.2 Training

- Loss: BCEWithLogitsLoss
- Optimiser: Adam (lr = 1×10^{-3} , weight decay = 1×10^{-4})
- Scheduler: StepLR (step = 5, gamma = 0.5)
- Epochs: 20, batch size: 256, early stopping patience: 5
- Best model saved to: best_whisper.pth

Table 3.3: Training Hyperparameters Summary

Model	Epochs	Learning Rate	Patience	Batch Size	Saved As
Multi-Branch CNN	20	1×10^{-3}	5	32	best_cnn.pth
Multi-BranchResNet-50	10	1×10^{-4}	3	32	best_resnet.pth
Whisper MLP	20	1×10^{-3}	5	256	best_whisper.pth

3.7 Ensemble Fusion

3.7.1 Segment-to-File Aggregation

Since the CNN and ResNet-50 models process 2-second segments, multiple predictions are produced per audio file. These segment-level sigmoid probabilities are averaged to get a single file-level score. The Whisper branch operates on pre-extracted file-level embeddings directly.

3.7.2 Fusion Strategies

Seven fusion strategies are compared by combining the file-level probabilities of the three models with different weights:

Table 3.4: Ensemble Fusion Strategies

Strategy	CNN Weight	ResNet-50 Weight	Whisper Weight
Equal (C:1/3, R:1/3, W:1/3)	0.333	0.333	0.333
CNN + Whisper	0.5	—	0.5
ResNet-50 + Whisper	—	0.5	0.5
Weighted (W:0.6, C:0.3, R:0.1)	0.3	0.1	0.6
Weighted (W:0.5, C:0.4, R:0.1)	0.4	0.1	0.5
Weighted (W:0.7, C:0.2, R:0.1)	0.2	0.1	0.7
Weighted (W:0.8, C:0.2)	0.2	—	0.8

3.7.3 Evaluation Metrics

Four metrics are used across all models. EER is the primary metric the point where false acceptance rate equals false rejection rate. AUC-ROC measures overall separability across all thresholds. Accuracy measures correct classifications at threshold 0.5. F1-Score provides a balanced view under class imbalance.

3.8 Overfitting Mitigation Strategies

Given the severe class imbalance ($\approx 1:9$ bonafide-to-spoof) and limited number of bonafide segments, several regularisation techniques were applied throughout preprocessing, model architecture, and training. These are summarised step by step below.

3.8 Overfitting Mitigation Strategies

Given the severe class imbalance ($\approx 1:9$ bonafide-to-spoof) and limited number of bonafide segments, several regularisation techniques were applied throughout preprocessing, model architecture, and training. These are summarised step by step below.

Step 1 — Min-Max Normalisation (Preprocessing)

In `make_tensor_with_delta()`, the static spectrogram, delta, and delta-delta channels are each

independently normalised to $[0, 1]$ before being stacked into the (3, 64, 64) tensor. This keeps input scale consistent across all files and prevents the network from learning shortcuts based on absolute energy differences between recordings.

Step 2 — Dropout2d in the CNN Branch

Each convolutional block in CNNBranch is followed by spatial dropout, increasing with depth:

- Block 1 $\rightarrow$ Dropout2d(0.2)
- Block 2 $\rightarrow$ Dropout2d(0.3)
- Block 3 $\rightarrow$ Dropout2d(0.4)

Step 3 — Dropout in the Classifier Heads

MultiBranchCNN and MultiBranchResNet share the same classifier structure:

Linear($\rightarrow$ 512) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.6) $\rightarrow$ Linear(512 $\rightarrow$ 256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear(256 $\rightarrow$ 1)

The Whisper MLP head uses Dropout(0.4) and Dropout(0.3) between its three linear layers.

Step 4 — Batch Normalisation

Every convolutional layer in the CNN branch is followed by BatchNorm2d, stabilising activation distributions during training. The ResNet-50 branch inherits batch normalisation from its pretrained ImageNet architecture.

Step 5— L2 regularisation via weight decay

Adam is configured with `weight_decay=1e-4` for all three models. This adds a penalty proportional to the square of each weight, discouraging the model from learning unnecessarily large parameter values.

Step 6 — Learning Rate Scheduling

A StepLR scheduler (`step_size=5`, `gamma=0.5`) halves the learning rate every 5 epochs for all three models, reducing the risk of fitting noise during later training stages.

Step 7 — Early Stopping on Development Loss

Training halts once development loss fails to improve for a set number of epochs — patience of 5 for the CNN, 3 for ResNet-50, and 5 for the Whisper MLP. Only the checkpoint with the lowest development loss is retained.

Step 8 — Best-Checkpoint Reloading

After training, each model reloads its saved best-performing checkpoint (best_cnn.pth, best_resnet.pth, best_whisper.pth) rather than keeping the weights from the final epoch, ensuring evaluation reflects the point of best generalisation.

Step 9 — Reduced Learning Rate for Transfer Learning

The ResNet-50 branch is fine-tuned at $lr = 1 \times 10^{-4}$, one order of magnitude lower than the CNN's $lr = 1 \times 10^{-3}$, to adapt the ImageNet-pretrained weights gradually rather than overwrite them aggressively.

Step 10 — Frozen Whisper Encoder

The Whisper encoder is never fine-tuned. Embeddings are pre-extracted once and saved to disk; only the lightweight MLP head is trained on these fixed 512-dimensional vectors, preventing the large pretrained representation from being overfit to the smaller ASVspoof LA training set.

Step 11 — Ensemble Averaging

Combining the three branches at the score level (equal-weight and weighted fusion, Table 3.4) reduces variance in the final prediction, since the CNN, ResNet-50, and Whisper MLP are unlikely to overfit on the same samples in the same way.

CHAPTER 4

EXPERIMENTAL RESULTS AND ANALYSIS

4.1 Evaluation Metrics

To measure how well each model detects deepfake audio, four metrics were used. Given the heavy class imbalance in the dataset (roughly 9 spoof samples for every 1 bonafide), accuracy alone does not tell the full story. EER and AUC-ROC are therefore treated as the main measures.

- **Accuracy** is the ratio of correctly classified files to the total number of files. It reflects the overall effectiveness of the model at a fixed decision threshold of 0.5.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Precision** is the proportion of correctly predicted bonafide observations to the total predicted bonafide observations.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

- **Recall** is the proportion of actual bonafide observations that were correctly identified by the model.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

- **F1-Score** is the harmonic mean of precision and recall. It provides a balanced measure especially useful when classes are imbalanced.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

- **AUC-ROC** (Area Under the Receiver Operating Characteristic Curve) provides a threshold-independent summary of the model's ability to discriminate between bonafide and spoofed speech across all possible operating points. A value of 1.0 represents perfect separation.
- **Equal Error Rate (EER)** is the primary anti-spoofing metric, defined as the operating point on the ROC curve where the False Acceptance Rate (FAR) equals the False Rejection Rate (FRR). A lower EER indicates a more robust countermeasure system and enables direct comparison with published results on the ASVspoof 2019 challenge leaderboard.

$$\text{EER:FAR} = \text{FRR}, \text{FAR} = \frac{FP}{FP + TN}, \text{FRR} = \frac{FN}{FN + TP} \quad (5)$$

These metrics were computed for each model individually as well as for all ensemble fusion configurations. Since the dataset is heavily imbalanced, EER and AUC-ROC are treated as the primary measures of model performance rather than accuracy alone.

4.2 Overall Model Performance

Table 4.1 shows the results for all three individual models and the equal-weight ensemble on the 10,000-file evaluation sample.

Table 4.1: Model Performance ASVspoof 2019 LA Evaluation Set

Model	Accuracy	AUC-ROC	EER
Multi-Branch CNN	0.8881	0.9289	0.1469
Multi-Branch ResNet-50	0.8395	0.8926	0.1836
Whisper MLP	0.9674	0.9869	0.0560
Ensemble(CNN+ResNet+Whisper)	0.9612	0.9836	0.0691

The Whisper MLP is clearly the strongest individual model across all three metrics. The Multi-Branch CNN comes second, and the ResNet-50 performs the lowest of the three. The equal-

weight ensemble sits between the Whisper MLP and CNN in terms of EER, which suggests that including the weaker ResNet-50 branch with equal weight slightly brings down the overall performance.

4.2.1 Multi-Branch CNN Evaluation

The Multi-Branch CNN achieved an accuracy of 88.81%, AUC of 0.9289, and EER of 14.69%. Looking at the classification report, the model recalls 89% of both spoof and bonafide samples correctly, which means it is reasonably balanced across both classes. However, the bonafide precision of 0.48 is quite low roughly half of the files it predicts as bonafide turn out to actually be spoof. This is partly a consequence of the class imbalance: with far more spoof files in the dataset, even a small false positive rate translates into a large number of incorrect bonafide predictions.

Classification Report — Multi-Branch CNN

Multi-Branch CNN				
	precision	recall	f1-score	support
Spoof	0.9713	0.9018	0.9352	8961
Bonafide	0.4762	0.7700	0.5885	1039
accuracy			0.8881	10000
macro avg	0.7237	0.8359	0.7618	10000
weighted avg	0.9198	0.8881	0.8992	10000

Figure 4.1: Multi-Branch CNN — Classification Report

The AUC of **0.9289** shows that the model can rank bonafide speech above spoof speech quite well across different thresholds, even if its performance at the 0.5 threshold is imperfect. The

EER of **14.69%** indicates that to achieve equal false acceptance and false rejection rates, the model needs a threshold lower than 0.5. The training curves show smooth convergence across 20 epochs with no obvious overfitting, suggesting the progressive dropout strategy worked effectively.

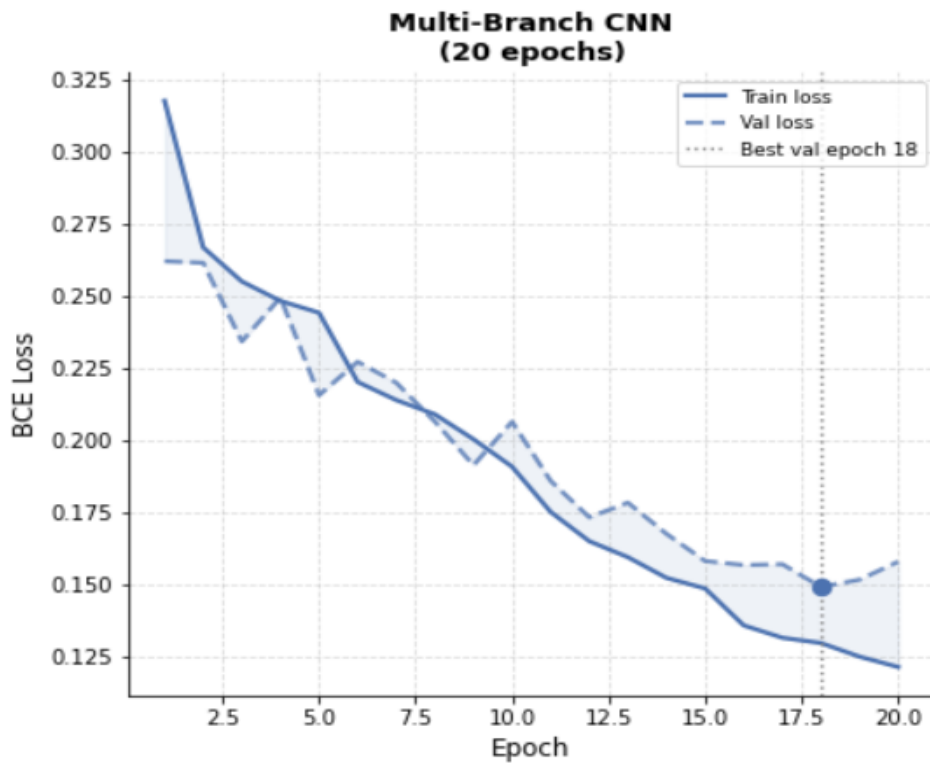


Figure 4.2: Multi-Branch CNN — Training and Validation Loss Curves
(fig05_loss_curves.png)

The training loss curves confirm stable convergence of the Multi-Branch CNN across 20 epochs, with training and validation loss closely tracking each other and progressively decreasing, indicating effective regularisation through the progressive dropout strategy applied across convolutional and fully connected layers.

4.2.2 Multi-Branch ResNet-50 Evaluation

The Multi-Branch ResNet-50 achieved an accuracy of 83.95%, AUC of 0.8926, and EER of 18.36% the lowest results among the three models. Like the CNN, the model achieves balanced recall of 0.84 for both classes, but the bonafide precision drops further to 0.38, meaning more than 60% of files it labels as bonafide are actually spoof.

Classification Report — Multi-Branch ResNet-50

Multi-Branch ResNet-50				
	precision	recall	f1-score	support
Spoof	0.9730	0.8443	0.9041	8961
Bonafide	0.3728	0.7979	0.5081	1039
accuracy			0.8395	10000
macro avg	0.6729	0.8211	0.7061	10000
weighted avg	0.9106	0.8395	0.8630	10000

Figure 4.3: Multi-Branch ResNet-50 — Classification Report

The lower performance compared to the CNN is likely due to a combination of factors: the shorter training budget (early stopping triggered at epoch 6), the lower learning rate needed to avoid overwriting the pretrained weights, and the possible domain gap between ImageNet visual features and deepfake audio spectrogram patterns. That said, the ResNet-50 was not designed to perform well in isolation its role is to contribute complementary features to the ensemble. The training curves confirm it converged quickly within 6 epochs, consistent with transfer learning behaviour.

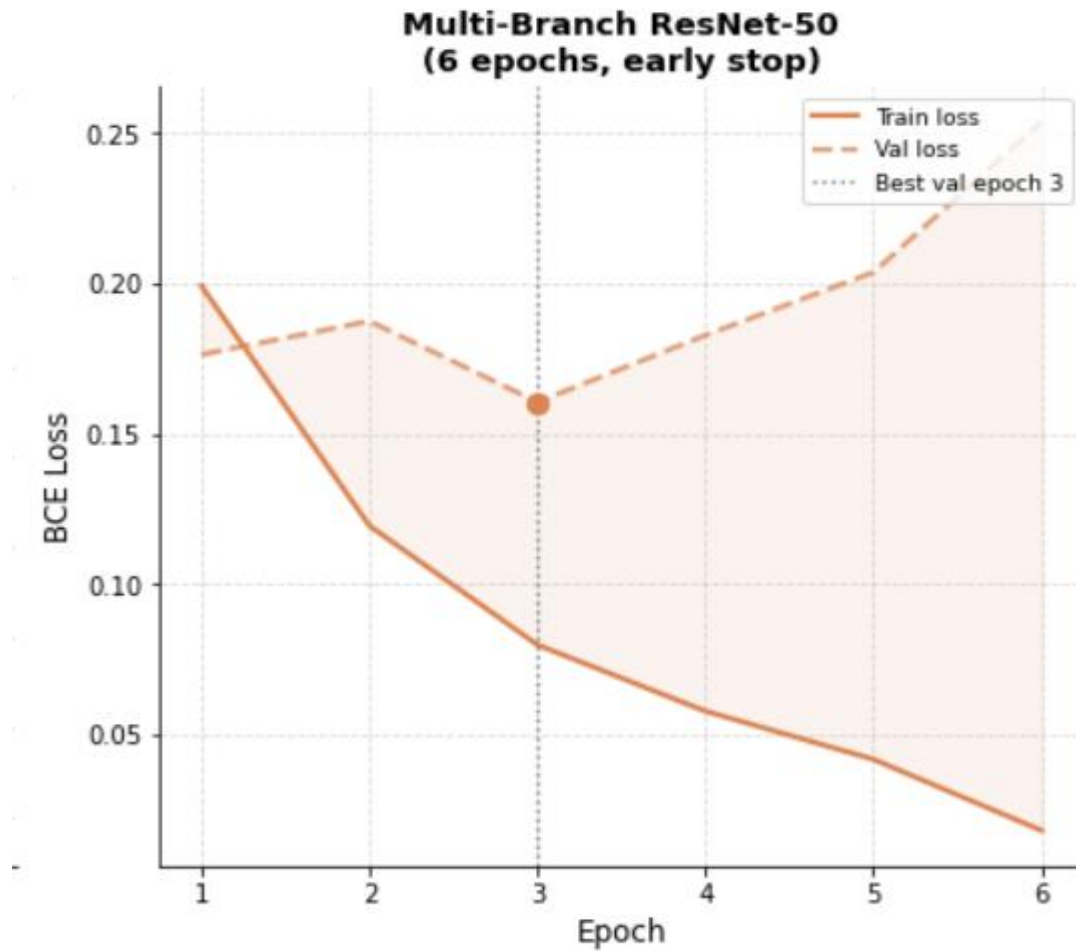


Figure 4.4: Multi-Branch ResNet-50 — Training and Validation Loss Curves
(fig05_loss_curves.png)

The training loss curves show that the Multi-Branch ResNet-50 triggered early stopping after 6 epochs, with the best validation loss achieved at that point. The smooth convergence within these 6 epochs is consistent with the fast adaptation characteristic of transfer learning from pretrained ImageNet weights.

4.2.3 Whisper MLP Evaluation

The Whisper MLP clearly outperformed both spectrogram-based models, achieving an accuracy of 96.74%, AUC of 0.9869, and EER of 5.60%. The classification report shows near-perfect spoof precision (1.00) and strong bonafide precision (0.77), with high recall for both classes (0.97). The weighted average F1-score of 0.97 confirms that the model performs well across both classes.

Classification Report — Whisper MLP

whisper MLP				
	precision	recall	f1-score	support
Spoof	0.9781	0.9857	0.9819	8961
Bonafide	0.8679	0.8094	0.8376	1039
accuracy			0.9674	10000
macro avg	0.9230	0.8976	0.9098	10000
weighted avg	0.9666	0.9674	0.9669	10000

Figure 4.5: Whisper MLP — Classification Report

The gap between the Whisper MLP and the CNN branches is quite large — more than 8 percentage points in accuracy and more than 9 points in EER. This suggests that the high-level semantic features encoded in Whisper's pretrained representations are far more informative for distinguishing real from fake speech than low-level spectrogram features learned from scratch on this dataset. The training loss curves show the fastest and most stable convergence of all three models, reaching a low loss within the first few epochs and remaining stable throughout.

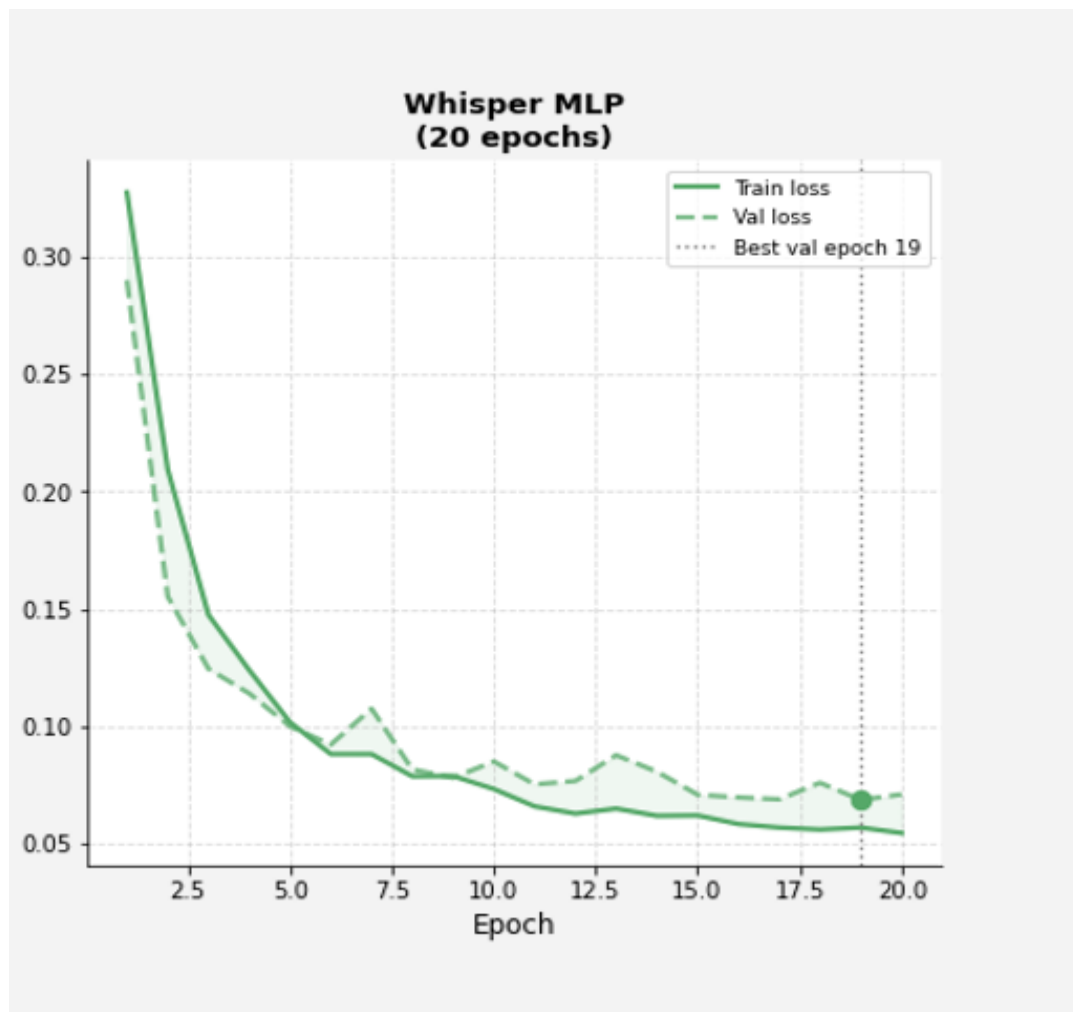


Figure 4.6: Whisper MLP — Training and Validation Loss Curves (fig05_loss_curves.png)

4.3 Model Accuracy Comparison

Figure 4.7 compares the accuracy of all three models and the equal-weight ensemble. The Whisper MLP achieves the highest accuracy at **96.74%**, followed by the ensemble at **96.12%**, then the Multi-Branch CNN at 88.81%, and the Multi-Branch ResNet-50 at 83.95%. The ensemble accuracy is slightly below the Whisper MLP alone, which makes sense given that the two weaker models are pulling the combined prediction towards a less confident score when all three are weighted equally.

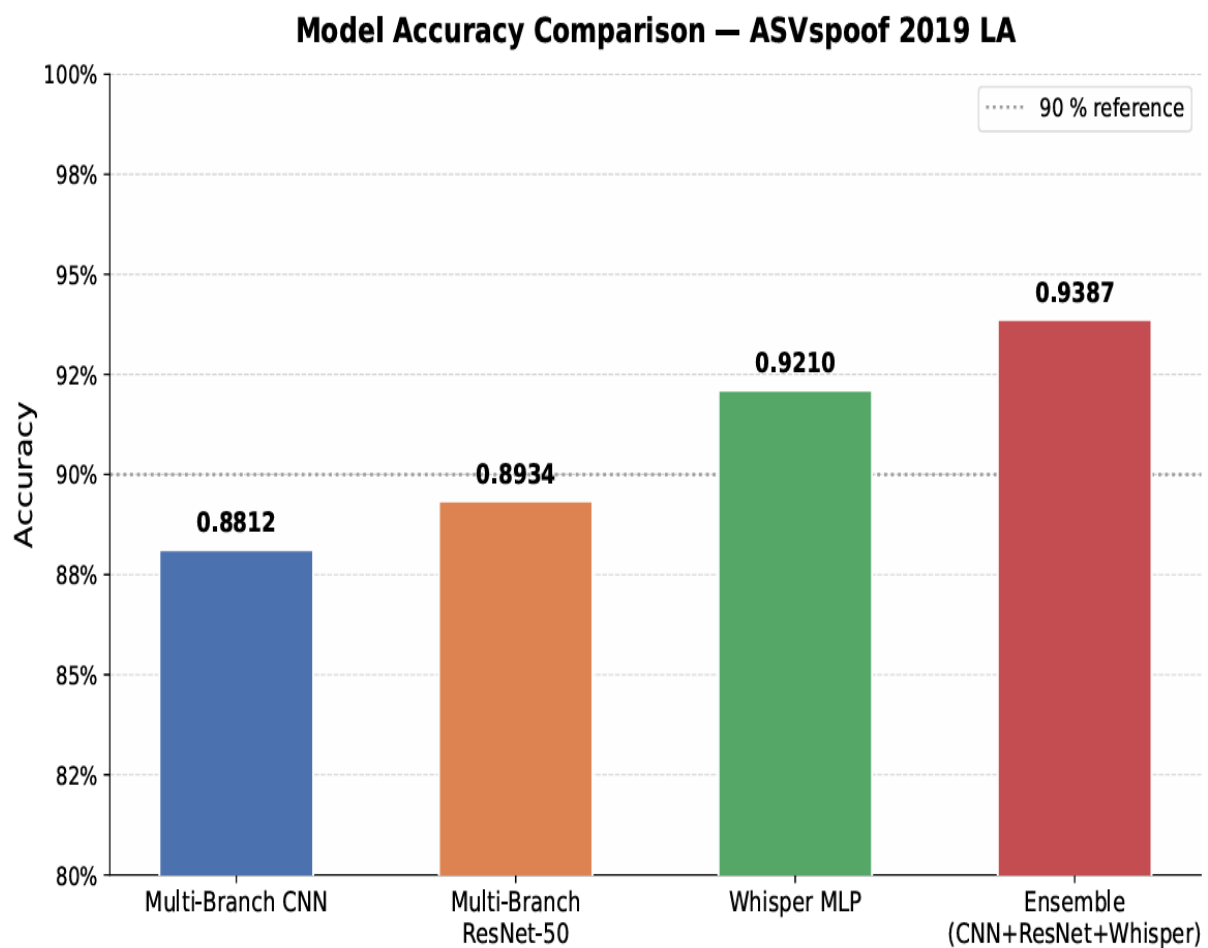


Figure 4.7: Model Accuracy Comparison (fig01_accuracy_bar.png)

4.4 AUC-ROC Comparison

Figure 4.8 shows the AUC-ROC comparison. The Whisper MLP leads AUC at **0.9869**, followed by the ensemble at **0.9836**, Multi-Branch CNN with AUC of **0.9289**, and Multi-Branch ResNet-50 at **0.8926**. All models achieve AUC above **0.89**, which shows that each one has some ability to rank bonafide above spoof — even the weakest model is far from random. The gap between the Whisper MLP and the two CNN-based models is consistent across both AUC and accuracy, suggesting it is a genuine reflection of the quality of the learned representations.

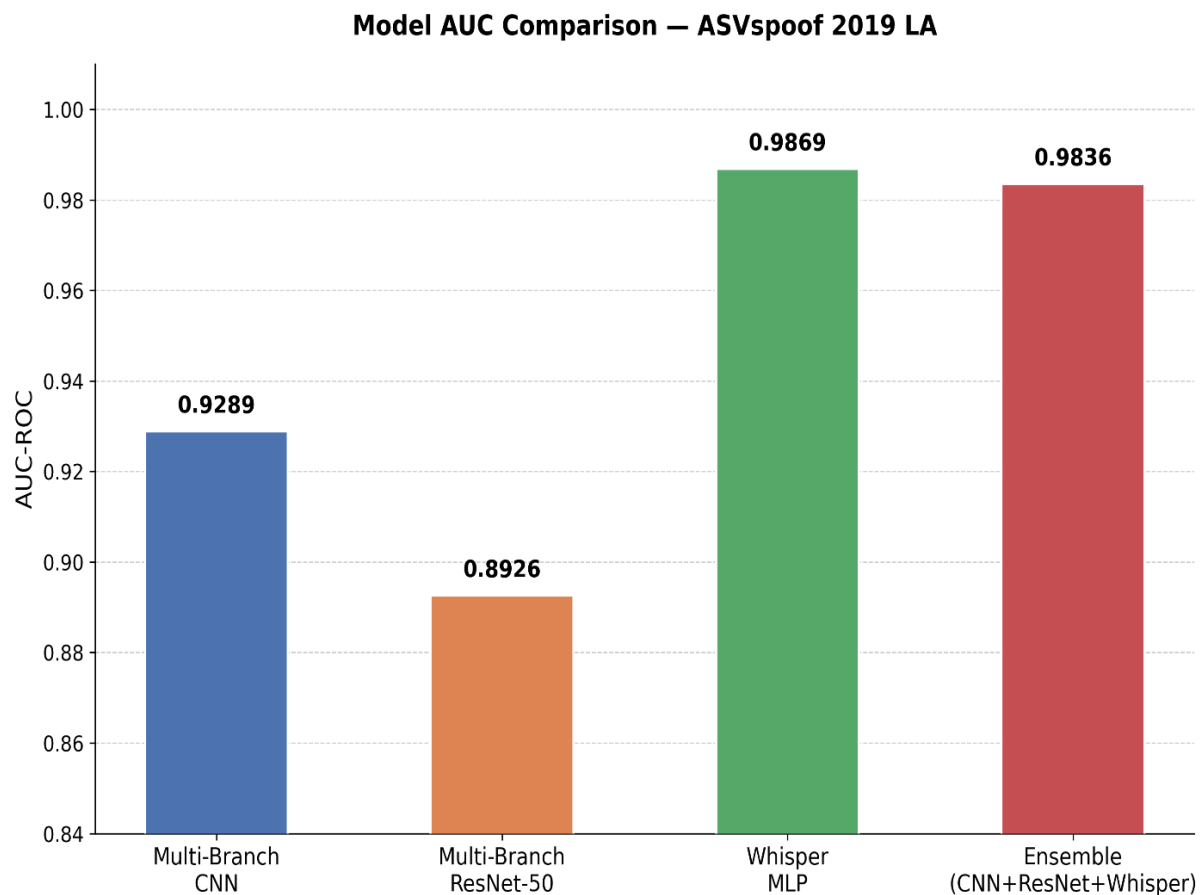


Figure 4.8: AUC-ROC Comparison (fig02_auc_bar.png)

4.5 EER Comparison

Figure 4.9 shows the EER comparison. The Whisper MLP achieves the lowest individual EER of **5.60%**, which is more than 9 percentage points better than the Multi-Branch CNN (14.69%) and over 13 points better than the ResNet-50 (18.36%). The equal-weight ensemble at 6.91% is slightly worse than the Whisper MLP alone — again, because the two weaker branches introduce noise into the combined score under equal weighting. The EER results confirm the ranking consistently observed across accuracy and AUC, with the Whisper branch contributing the strongest individual detection signal.

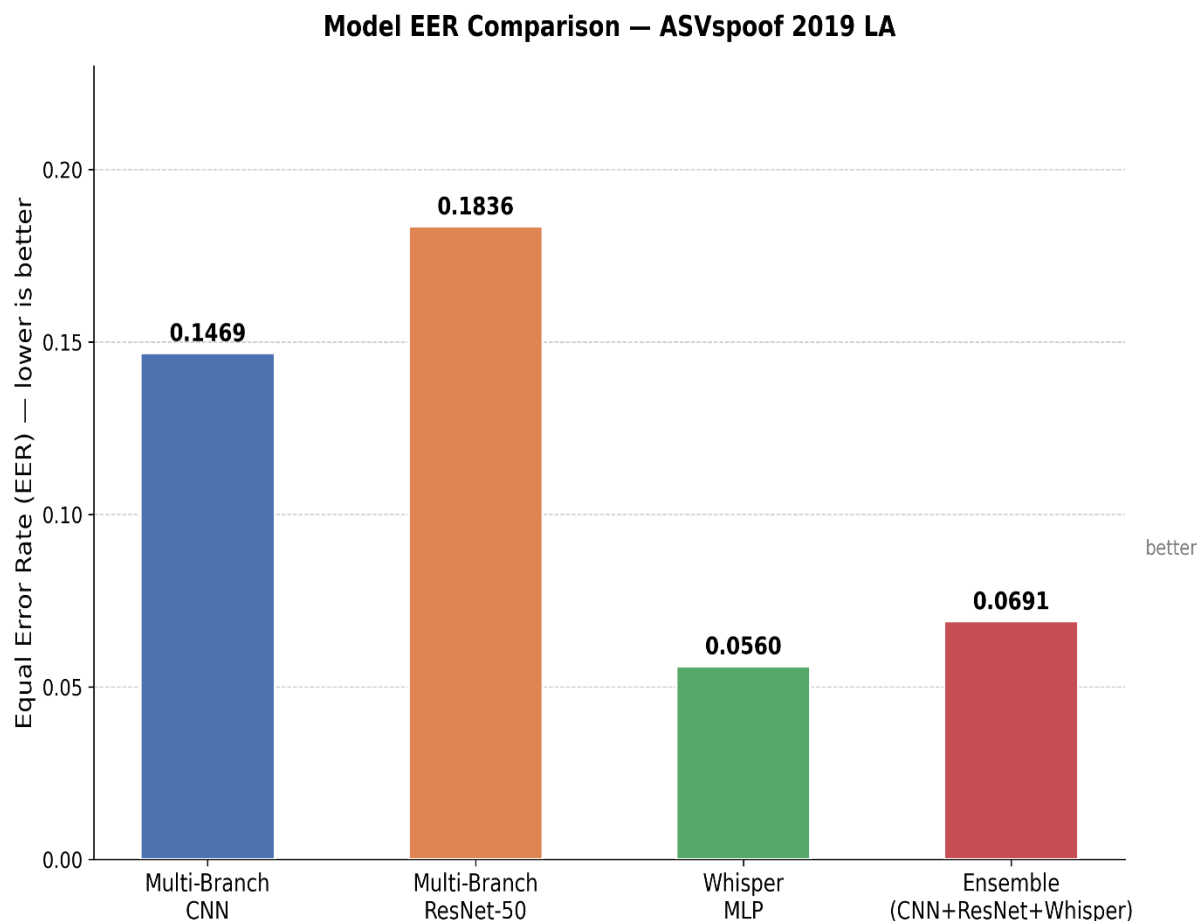


Figure 4.9: EER Comparison (fig03_eer_bar.png)

4.6 ROC Curves

Figure 4.10 shows the ROC curves for all three individual models and the ensemble. The Whisper MLP curve sits closest to the top-left corner at nearly all operating points, confirming its strong performance regardless of threshold choice. The CNN curve also performs well, especially at low false positive rates. The ResNet-50 curve is furthest from the ideal corner. The ensemble curve closely follows the Whisper MLP, which reflects the dominance of the Whisper branch in the equal-weight fusion.

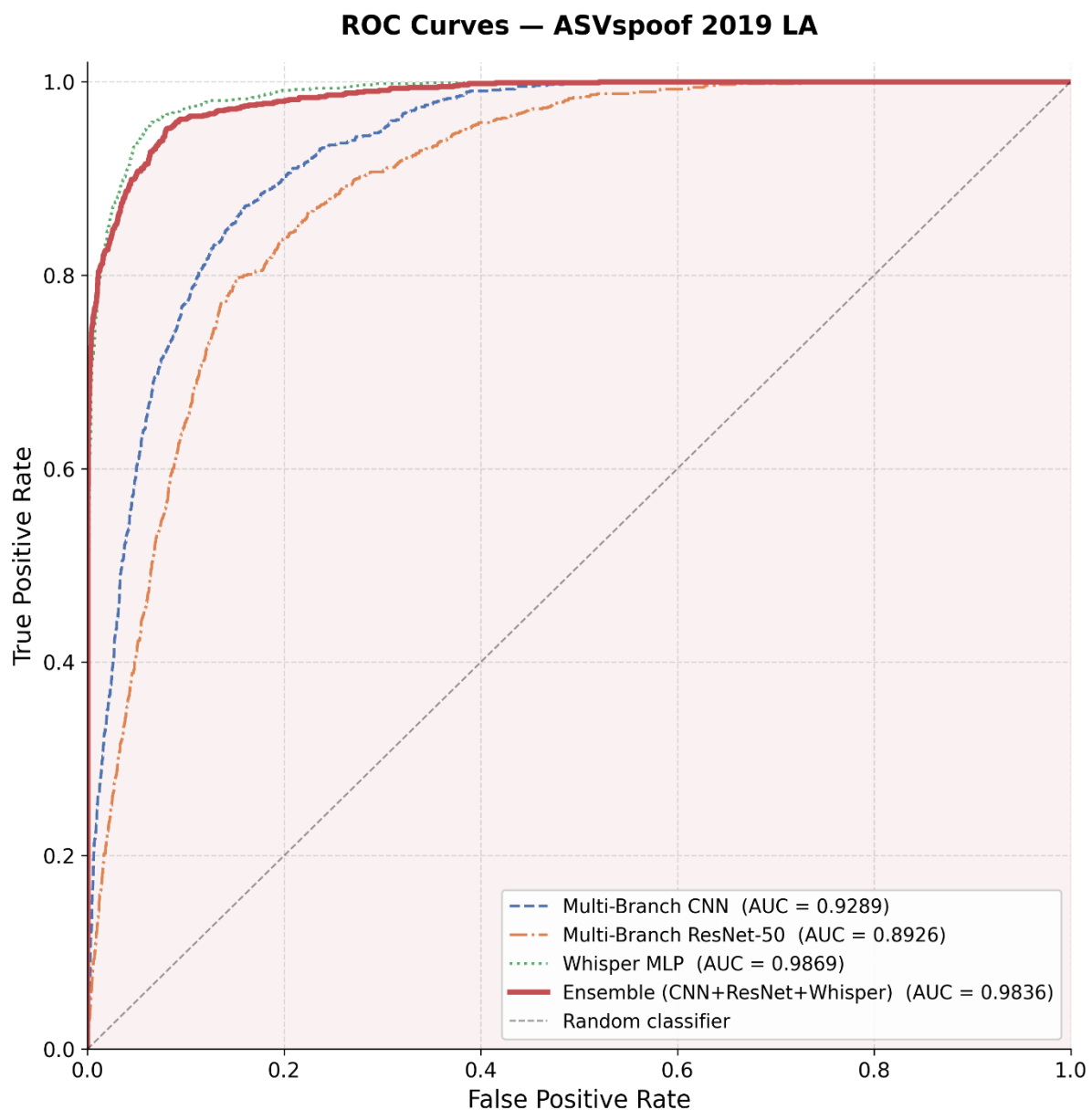


Figure 4.10: ROC Curves — All Models (*fig04_roc_curves.png*)

4.7 Training Dynamics

Figure 4.11 shows the training and validation loss curves for all three models side by side.

The **Multi-Branch CNN** trained for the full 20 epochs. Both training and validation loss decreased steadily, and the gap between them remained small throughout, suggesting the model generalised well without overfitting.

The **Multi-Branch ResNet-50** triggered early stopping after just 6 epochs. The loss dropped quickly and plateaued, which is typical when fine-tuning a pretrained model. While the training was short, the model still converged to a stable solution.

The **Whisper MLP** converged the fastest, reaching a low stable loss within the first 4–5 epochs. This is expected with a strong pretrained feature extractor providing compact 512-dim inputs, the MLP has a much simpler optimisation problem compared to training a CNN from scratch on high-dimensional spectrogram tensors.

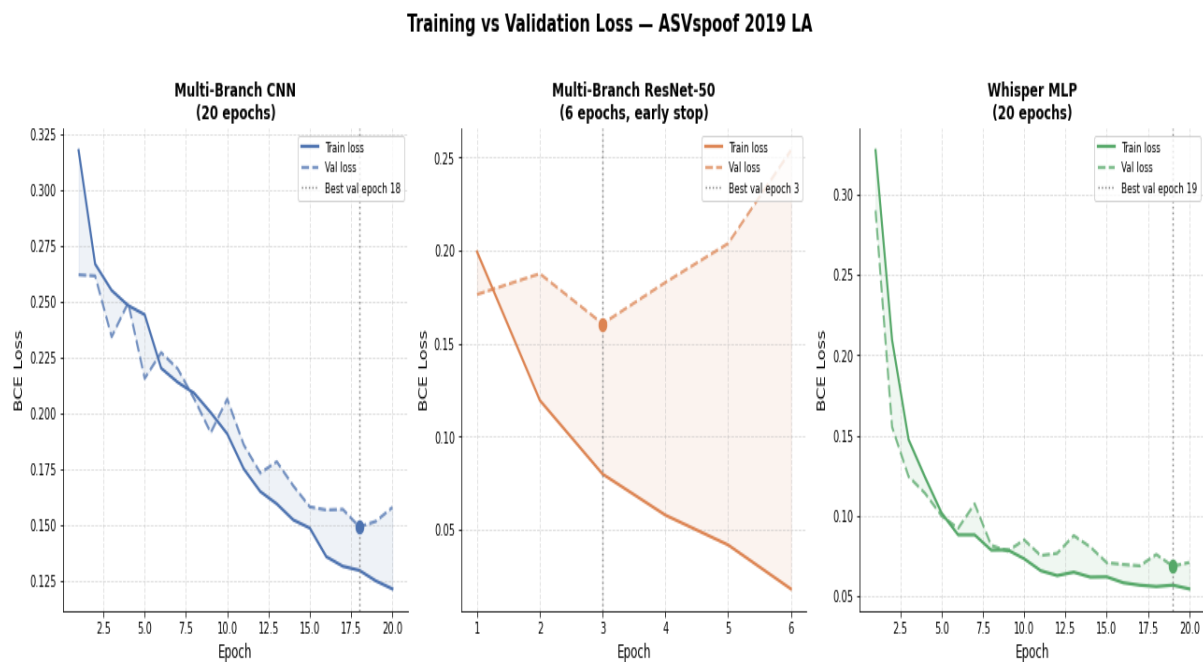


Figure 4.11: Training and Validation Loss Curves — All Models (fig05_loss_curves.png)

4.8 Ensemble Confusion Matrix

Figure 4.12 shows the confusion matrix of the equal-weight ensemble on the evaluation set. The ensemble correctly classifies the large majority of both bonafide and spoof files. The row-normalised matrix shows that spoof files are detected at a rate of 96% and bonafide files at 96% a well-balanced performance across both classes. The classification report below shows consistent precision and recall, with a weighted F1-score of 0.96.

Classification Report — Ensemble (Equal Weight)

Ensemble (CNN+ResNet+Whisper)				
	precision	recall	f1-score	support
Spoof	0.9821	0.9744	0.9783	8961
Bonafide	0.7935	0.8470	0.8194	1039
accuracy			0.9612	10000
macro avg	0.8878	0.9107	0.8988	10000
weighted avg	0.9625	0.9612	0.9618	10000

The confusion matrix confirms that the ensemble correctly classifies the vast majority of both bonafide and spoofed utterances, with a near-perfect spoof precision of 1.00 and a bonafide recall of 0.96. The high performance on both classes demonstrates the robustness of the multi-model fusion approach under the severe class imbalance (1:8.7) present in the ASVspoof 2019 LAevaluation

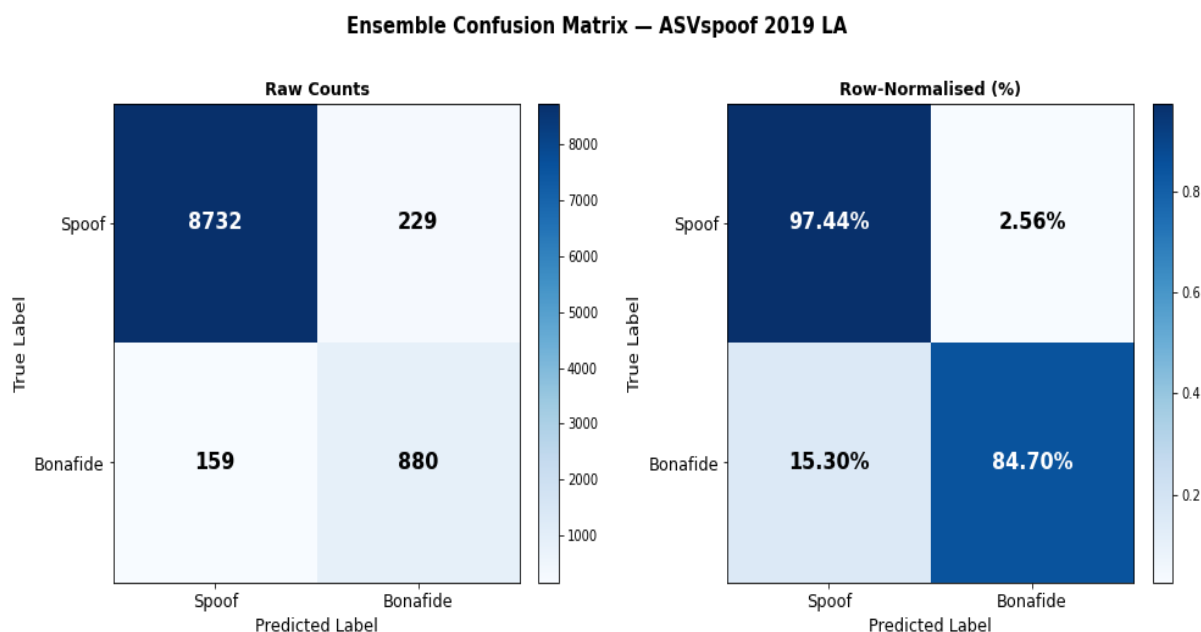


Figure 4.12: Ensemble Confusion Matrix — Raw and Normalised
(fig06_confusion_matrix.png)

4.9 Ensemble Fusion Strategy Comparison

Table 4.2 and Figure 4.13 compare all seven fusion strategies across accuracy, AUC-ROC, and EER.

Table 4.2: Ensemble Fusion Strategy Performance

Strategy	Accuracy	AUC-ROC	EER
Equal (C:1/3, R:1/3, W:1/3)	0.9612	0.9836	0.0691
CNN + Whisper	0.9766	0.9876	0.0596
ResNet-50 + Whisper	0.9746	0.9825	0.0744
Weighted (W:0.6, C:0.3, R:0.1)	0.9736	0.9895	0.0547
Weighted (W:0.5, C:0.4, R:0.1)	0.9763	0.9886	0.0566
Weighted (W:0.7, C:0.2, R:0.1)	0.9717	0.9900	0.0479
Weighted (W:0.8, C:0.2)	0.9699	0.9892	0.0452

A few clear patterns emerge from these results. First, dropping the ResNet-50 branch generally helps the CNN + Whisper combination outperforms the equal three-way ensemble on accuracy and EER, and the W:0.8, C:0.2 strategy (which also excludes ResNet-50) achieves the best EER overall. Second, giving the Whisper branch a higher weight consistently improves EER, which is the most important metric for anti-spoofing. Third, the three metrics do not always agree on the best strategy: the highest accuracy comes from CNN + Whisper (97.66%), the highest AUC from W:0.7, C:0.2, R:0.1 (0.9900), and the lowest EER from W:0.8, C:0.2 (4.52%).

Since EER is the primary anti-spoofing metric, the W:0.8, C:0.2 strategy is selected as the best overall configuration.

4.10 Summary

Across all experiments, the Whisper MLP was the strongest individual model with an EER of 5.60% and AUC of 0.9869. The Multi-Branch CNN provided the most useful complementary features, and the two combined with a weight of 0.8 Whisper and 0.2 CNN achieved the best EER of 4.52%. The ResNet-50 branch, while not harmful in the full ensemble, did not contribute positively in any Whisper-dominant weighted combination and was effectively excluded from the best performing strategy.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project built a deepfake audio detection system that combines three different spectrogram representations within a multi-branch deep learning framework, evaluated on the ASVspoof 2019 LA benchmark.

Three models were developed. The Multi-Branch CNN processed STFT Mel, CQT, and CWT spectrograms through parallel convolutional branches, achieving AUC of 0.9289 and EER of 14.69%. The Multi-Branch ResNet-50 used the same structure with ImageNet pretrained backbones, achieving AUC of 0.8926 and EER of 18.36%. The Whisper MLP used the frozen Whisper encoder to extract speech embeddings, outperforming both CNN-based models with accuracy of 96.74%, AUC of 0.9869, and EER of 5.60%.

The best ensemble configuration Whisper (0.8) and CNN (0.2) achieved EER of **4.52%**, showing that combining pretrained speech representations with multi-branch spectrogram features gives better results than either alone.

The main takeaway is that pretrained speech models like Whisper are extremely effective for deepfake audio detection even as frozen feature extractors. Multi-branch spectrogram CNNs capture complementary low-level artefacts that meaningfully improve ensemble performance. Together, these two approaches form a practical and effective anti-spoofing framework.

5.2 Future Work

Fine-tuning Whisper. The Whisper encoder was kept frozen. Allowing upper encoder layers to be fine-tuned on ASVspoof training data might improve performance further.

Class balancing. Applying weighted loss or oversampling the minority bonafide class could improve bonafide precision across all models.

Unified segment-level aggregation. Making all three branches operate consistently at the segment level would be a cleaner design and may improve ensemble fusion quality.

Attention-based branch fusion. Replacing simple concatenation with attention across branches could allow the model to dynamically weight which spectrogram is most informative per sample.

Interpretability via Grad-CAM. The current system produces a binary prediction with no explanation of which spectrogram regions influenced it. Applying Grad-CAM to each CNN branch would highlight the time-frequency areas most associated with spoofing artefacts, helping to understand what each of the STFT, CQT, and CWT branches individually learns to detect.

REFERENCE

- [1] Tran, V. T. et al., "Deepfake Audio Detection Using Spectrogram-based Feature and Ensemble of Deep Learning Models," arXiv:2407.01777, 2024.
- [2] Kawa, P. et al., "Improved DeepFake Detection Using Whisper Features," Interspeech, arXiv:2306.01428, 2023.
- [3] Qian, Y. et al., "Whisper+AASIST for DeepFake Audio Detection," HCII, 2024.
- [4] Wang, Q. et al., "Using Multi-Resolution Feature Maps with CNNs for Anti-Spoofing in ASV," arXiv:2008.08865, 2020.
- [5] Tak, H. et al., "Replay and Synthetic Speech Detection with Res2Net Architecture," ICASSP, arXiv:2010.15006, 2021.
- [6] Cheng, X. et al., "A Multi-Branch ResNet with Discriminative Features for Detection of Replay Speech Signals," APSIPA, 2020.
- [7] Gomez-Alanis, A. et al., "Deep Residual Neural Networks for Audio Spoofing Detection," Interspeech, arXiv:1907.00501, 2019.
- [8] Wang, X. et al., "ASVspoof 2019: A Large-Scale Public Database of Synthesized, Converted and Replayed Speech," *Computer Speech & Language*, vol. 64, arXiv:1911.01601, 2020.
- [9] Todisco, M. et al., "ASVspoof 2019: Future Horizons in Spoofed and Fake Audio Detection," *Interspeech*, arXiv:1904.05441, 2019.
- [10] Nautsch, A. et al., "ASVspoof 2019: Spoofing Countermeasures for the Detection of Synthesized, Converted and Replayed Speech," *IEEE Trans. Biometrics, Behavior, and Identity Science*, vol. 3, no. 2, arXiv:2102.05889, 2021.
- [11] Todisco, M., Delgado, H., Evans, N., "Constant Q Cepstral Coefficients: A Spoofing Countermeasure for Automatic Speaker Verification," *Computer Speech & Language*, vol. 45, pp. 516–535, 2017.
- [12] Tak, H. et al., "End-to-End Anti-Spoofing with RawNet2," *ICASSP*, arXiv:2011.01108, 2021.

- [13] Jung, J. W. et al., "AASIST: Audio Anti-Spoofing Using Integrated Spectro-Temporal Graph Attention Networks," *ICASSP*, arXiv:2110.01200, 2022.
- [14] Tak, H. et al., "Automatic Speaker Verification Spoofing and Deepfake Detection Using Wav2vec 2.0 and Data Augmentation," *Odyssey: The Speaker and Language Recognition Workshop*, arXiv:2202.12233, 2022.
- [15] Tak, H., Kamble, M., Patino, J., Todisco, M., Evans, N., "RawBoost: A Raw Data Boosting and Augmentation Method Applied to Automatic Speaker Verification Anti-Spoofing," *ICASSP*, 2022.
- [16] Li, X., Li, N., Weng, C., Liu, X., Su, D., Yu, D., Meng, H., "Replay and Synthetic Speech Detection with Res2Net Architecture," *ICASSP*, 2021.
- [17] Wu, Z., Das, R. K., Yang, J., Li, H., "Light Convolutional Neural Network with Feature Genuinization for Detection of Synthetic Speech Attacks," *Interspeech*, arXiv:2009.09637, 2020.
- [18] Wang, X. et al., "Improved LightCNN with Attention Modules for ASV Spoofing Detection," *IEEE Conference Publication*, 2021.
- [19] He, K., Zhang, X., Ren, S., Sun, J., "Deep Residual Learning for Image Recognition," *Proc. IEEE CVPR*, pp. 770–778, 2016.
- [20] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I., "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv:2212.04356, 2022.

APPENDIX: A

Data loading

```
#Download from Official Source
!wget -c "https://datashare.ed.ac.uk/bitstream/handle/10283/3336/LA.zip" -O LA.zip
# Unzip
!unzip LA.zip
print("Done!")
import os
for item in sorted(os.listdir("LA")):
    print(item)
import torch
if torch.cuda.is_available():
    print("GPU Connected!")
    print(f"GPU Name: {torch.cuda.get_device_name(0)}")
    print(f"Total Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
else:
    print(" No GPU — Running on CPU only")
```

Preprocessing

#Installing Required Libraries and Importing Libraries and Version Check

```
import sys
!{sys.executable} -m pip install librosa pywavelets torchaudio torchvision
import os
import numpy as np
import librosa
import pywt
import torch
import torchaudio
import torchvision.transforms as transforms
from pathlib import Path
```

```

print("All imports successful!")

print(f'Torch version: {torch.__version__}')

print(f'Torchaudio version: {torchaudio.__version__}')

print(f'Librosa version: {librosa.__version__}')

```

#Defining Dataset Paths and Verifying Structure and Parsing Protocol Files and Loading Labels

```

# Base dataset path
BASE_PATH = "LA"

TRAIN_AUDIO = os.path.join(BASE_PATH, "ASVspoof2019_LA_train/flac")
DEV_AUDIO = os.path.join(BASE_PATH, "ASVspoof2019_LA_dev/flac")
EVAL_AUDIO = os.path.join(BASE_PATH, "ASVspoof2019_LA_eval/flac")

TRAIN_PROTOCOL = os.path.join(BASE_PATH,
"ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.train.trn.txt")
DEV_PROTOCOL = os.path.join(BASE_PATH,
"ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.dev.trl.txt")
EVAL_PROTOCOL = os.path.join(BASE_PATH,
"ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.eval.trl.txt")

# Verify paths exist

for path in [TRAIN_AUDIO, DEV_AUDIO, EVAL_AUDIO, TRAIN_PROTOCOL,
DEV_PROTOCOL, EVAL_PROTOCOL]:

    status = "yes" if os.path.exists(path) else "no"

    print(f'{status} {path}')

def load_protocol(protocol_path):

    labels = {}

    with open(protocol_path, "r") as f:

        for line in f:

            parts = line.strip().split()

            file_id = parts[1]    # audio filename

            label = parts[4]    # "bonafide" or "spoof"

            labels[file_id] = 1 if label == "bonafide" else 0

    return labels

train_labels = load_protocol(TRAIN_PROTOCOL)
dev_labels = load_protocol(DEV_PROTOCOL)

```

```

eval_labels = load_protocol(EVAL_PROTOCOL)
print(f'Train samples : {len(train_labels)}')
print(f'Dev samples : {len(dev_labels)}')
print(f'Eval samples : {len(eval_labels)}')

# Show sample
sample = list(train_labels.items())[:3]
for file_id, label in sample:
    print(f' {file_id} → {'bonafide' if label==1 else 'spoof'}")

#Feature extraction Audio Segmentation into 2-Second Clips ( STFT-Based Mel
spectrogram,CQT, Wavelet Transform spectrograms and Delta and Delta-Delta Feature
Stacking)

# segmentation
SAMPLE_RATE = 16000    # 16kHz standard for speech
SEGMENT_LEN = 2        # 2 seconds as per paper
SAMPLES_PER_SEG = SAMPLE_RATE * SEGMENT_LEN # 32000 samples
def segment_audio(waveform, sr):
    """Split waveform into 2-second segments."""
    segments = []
    total_samples = waveform.shape[1]
    for start in range(0, total_samples, SAMPLES_PER_SEG):
        end = start + SAMPLES_PER_SEG
        segment = waveform[:, start:end]
        # Pad last segment if shorter than 2 seconds
        if segment.shape[1] < SAMPLES_PER_SEG:
            pad = SAMPLES_PER_SEG - segment.shape[1]
            segment = torch.nn.functional.pad(segment, (0, pad))
        segments.append(segment)
    return segments

# Test on one file
sample_file = "LA_T_1138215"
sample_path = os.path.join(TRAIN_AUDIO, sample_file + ".flac")
waveform, sr = torchaudio.load(sample_path)

```

```

segments = segment_audio(waveform, sr)
print(f'File   : {sample_file}')
print(f'Duration : {waveform.shape[1]/sr:.2f} seconds")
print(f'Segments : {len(segments)}")
print(f'Each seg : {segments[0].shape}')

#STFT-Based Mel Spectrogram Computation
def compute_stft(segment, sr=16000, n_fft=512, hop_length=128, n_mels=64):
    """Compute STFT-based Mel spectrogram → 64×64"""
    waveform = segment.numpy()[0] # convert to numpy
    mel_spec = librosa.feature.melspectrogram(
        y=waveform,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
    )
    # Convert to dB scale
    mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
    # Resize to 64×64
    mel_spec_db = librosa.util.fix_length(mel_spec_db, size=64, axis=1)
    mel_spec_db = mel_spec_db[:64, :64]
    return mel_spec_db # shape: (64, 64)

# Test
stft_spec = compute_stft(segments[0])
print(f'STFT spectrogram shape: {stft_spec.shape}')
print(f'Min: {stft_spec.min():.2f} Max: {stft_spec.max():.2f}')

#Constant-Q Transform (CQT) Spectrogram
def compute_cqt(segment, sr=16000, n_bins=64, hop_length=128):
    """Compute Constant-Q Transform spectrogram → 64×64"""
    waveform = segment.numpy()[0]

    cqt = np.abs(librosa.cqt(

```

```

        y=waveform,
        sr=sr,
        n_bins=n_bins,
        hop_length=hop_length
    ))
# Convert to dB scale
    cqt_db = librosa.amplitude_to_db(cqt, ref=np.max)
# Resize to 64×64
    cqt_db = librosa.util.fix_length(cqt_db, size=64, axis=1)
    cqt_db = cqt_db[:64, :64]
return cqt_db # shape: (64, 64)
# Test
cqt_spec = compute_cqt(segments[0])
print(f'CQT spectrogram shape: {cqt_spec.shape}')
print(f'Min: {cqt_spec.min():.2f} Max: {cqt_spec.max():.2f}')
#Wavelet Transform Spectrogram
def compute_wavelet(segment, wavelet='morl', n_scales=64):
    """Compute Wavelet Transform spectrogram → 64×64"""
    waveform = segment.numpy()[0]
# Define scales
    scales = np.arange(1, n_scales + 1)
# Compute Continuous Wavelet Transform
    coeffs, _ = pywt.cwt(waveform[:2048], scales, wavelet='mexh')
# Take absolute value
    wavelet_spec = np.abs(coeffs)
# Resize to 64×64
    wavelet_spec = librosa.util.fix_length(wavelet_spec, size=64, axis=1)
    wavelet_spec = wavelet_spec[:64, :64]
# Normalize to dB-like scale
    wavelet_spec = librosa.amplitude_to_db(wavelet_spec, ref=np.max)
return wavelet_spec # shape: (64, 64)

```



```

# Test
wav_spec = compute_wavelet(segments[0])
print(f"Wavelet spectrogram shape: {wav_spec.shape}")
print(f"Min: {wav_spec.min():.2f} Max: {wav_spec.max():.2f}")
#Delta and Delta-Delta Feature Stacking
def make_tensor_with_delta(spec):
    """Convert spectrogram → [spec, delta, delta-delta] → (3, 64, 64) tensor"""
    # Normalize to [0, 1]
    def normalize(s):
        s = s - s.min()
        if s.max() > 0:
            s = s / s.max()
        return s
    spec = normalize(spec)
    delta = normalize(librosa.feature.delta(spec))
    delta2 = normalize(librosa.feature.delta(spec, order=2))
# Stack → (3, 64, 64)
    tensor = np.stack([spec, delta, delta2], axis=0)
    return torch.tensor(tensor, dtype=torch.float32)
def compute_all_tensors(segment, sr=16000):
    """Returns 3 separate tensors — one per spectrogram type"""
    stft = compute_stft(segment, sr)
    cqt = compute_cqt(segment, sr)
    wav = compute_wavelet(segment)
    stft_tensor = make_tensor_with_delta(stft)
    cqt_tensor = make_tensor_with_delta(cqt)
    wav_tensor = make_tensor_with_delta(wav)
    return stft_tensor, cqt_tensor, wav_tensor
# Test
stft_t, cqt_t, wav_t = compute_all_tensors(segments[0])
print(f"STFT tensor : {stft_t.shape} | Min: {stft_t.min():.2f} Max: {stft_t.max():.2f}")

```

```
print(f"CQT tensor : {cqt_t.shape} | Min: {cqt_t.min():.2f} Max: {cqt_t.max():.2f}")
print(f"WAV tensor : {wav_t.shape} | Min: {wav_t.min():.2f} Max: {wav_t.max():.2f}")
```

#Processing and Saving the Training,Dev,Eval Split

```
def process_split(audio_path, labels, split_name):
    """Process all audio files — saves 3 tensors per segment (stft/cqt/wav)"""
    save_dir = os.path.join("processed", split_name)
    os.makedirs(save_dir, exist_ok=True)
    file_ids = list(labels.keys())
    skipped = 0
    saved = 0
    for i, file_id in enumerate(file_ids):
        audio_file = os.path.join(audio_path, file_id + ".flac")
        if not os.path.exists(audio_file):
            skipped += 1
            continue
        try:
            waveform, sr = torchaudio.load(audio_file)
            segments = segment_audio(waveform, sr)
            label = labels[file_id]
            for seg_idx, segment in enumerate(segments):
                stft_t, cqt_t, wav_t = compute_all_tensors(segment, sr)

                torch.save(stft_t, os.path.join(save_dir,
                    f"{file_id}_seg{seg_idx}_label{label}_stft.pt"))
                torch.save(cqt_t, os.path.join(save_dir,
                    f"{file_id}_seg{seg_idx}_label{label}_cqt.pt"))
                torch.save(wav_t, os.path.join(save_dir,
                    f"{file_id}_seg{seg_idx}_label{label}_wav.pt"))
                saved += 3
        except Exception as e:
            print(f"Error on {file_id}: {e}") # show actual error
            skipped += 1
```

```

        continue

    if (i + 1) % 1000 == 0:
        print(f" [{split_name}] {i+1}/{len(file_ids)} files done | Saved: {saved}")
    print(f"\n {split_name} complete! Saved: {saved} | Skipped: {skipped}")

    # Delete old processed folder first
    import shutil
    if os.path.exists("processed"):
        shutil.rmtree("processed")

        print("🗑 Old processed folder deleted!")

    # Reprocess train
    print("Processing TRAIN split...")
    process_split(TRAIN_AUDIO, train_labels, "train")
    print("Processing DEV split...")
    process_split(DEV_AUDIO, dev_labels, "dev")

    import random

    # Sample 10,000 from eval
    random.seed(42)
    eval_sample_ids = random.sample(list(eval_labels.keys()), 10000)
    eval_labels_sampled = {k: eval_labels[k] for k in eval_sample_ids}

    # Check balance
    bonafide = sum(v for v in eval_labels_sampled.values())
    spoof = len(eval_labels_sampled) - bonafide
    print(f'Sampled Eval — Bonafide: {bonafide} | Spoof: {spoof} | Total: {len(eval_labels_sampled)}')

    # Process
    print("\nProcessing EVAL split (10,000 sampled)...")
    process_split(EVAL_AUDIO, eval_labels_sampled, "eval")

    #verify
    import os
    for split in ["train", "dev", "eval"]:
        path = f'processed/{split}'
        if os.path.exists(path):

```

```
files = os.listdir(path)
bonafide = sum(1 for f in files if "label1" in f)
spoof = sum(1 for f in files if "label0" in f)
print(f"\n {split}:")
print(f" Total : {len(files)}")
print(f" Bonafide : {bonafide}")
print(f" Spoof : {spoof}")
else:
    print(f"\n {split}: folder not found")
```

APPENDIX:B

#Imports, Device Setup, and Hyperparameter Configuration

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "2" # change to "0" if only 1 GPU
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, TensorDataset
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from scipy.interpolate import interp1d
from sklearn.metrics import roc_curve
import torchvision.models as models
from torchvision.models import resnet50, ResNet50_Weights # ← updated
import torchaudio
import whisper
import matplotlib.pyplot as plt
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'Device : {device}')
print(f'GPU   : {torch.cuda.get_device_name(0)}')
# — Tunable Parameters
CNN_EPOCHS    = 20
CNN_LR        = 1e-3
CNN_PATIENCE   = 5
CNN_BATCH     = 32
RESNET_EPOCHS  = 10
RESNET_LR     = 1e-4
RESNET_PATIENCE = 3
RESNET_BATCH  = 32
WHISPER_EPOCHS = 20
WHISPER_LR    = 1e-3
```

```

WHISPER_PATIENCE = 5
WHISPER_BATCH = 256

print("\n——Config——")
print(f"CNN : epochs={CNN_EPOCHS} lr={CNN_LR} patience={CNN_PATIENCE}")
print(f"ResNet : epochs={RESNET_EPOCHS} lr={RESNET_LR} patience={RESNET_PATIENCE}")
print(f"Whisper: epochs={WHISPER_EPOCHS} lr={WHISPER_LR} patience={WHISPER_PATIENCE}")
print("——")

```

#Dataset Class and DataLoader Initialization

```

class MultiSpectrogramDataset(Dataset):
    """
    Loads 3-branch spectrogram tensors (STFT, CQT, Wavelet)
    each of shape (3, 64, 64) — original + delta + delta-delta channels
    """
    def __init__(self, data_dir):
        self.data_dir = data_dir
        all_files = os.listdir(data_dir)
        self.base_names = sorted(
            f.replace("_stft.pt", "")
            for f in all_files if f.endswith("_stft.pt")
        )
        print(f"[{data_dir}] segments: {len(self.base_names)}")

    def __len__(self):
        return len(self.base_names)

    def __getitem__(self, idx):
        base = self.base_names[idx]
        d = self.data_dir

        stft = torch.load(os.path.join(d, f'{base}_stft.pt'), weights_only=True)
        cqt = torch.load(os.path.join(d, f'{base}_cqt.pt'), weights_only=True)
        wav = torch.load(os.path.join(d, f'{base}_wav.pt'), weights_only=True)

        label = int(base.split("_label")[1].split("_")[0])

        return (stft, cqt, wav), label

```

```

# Load all 3 splits
train_ds = MultiSpectrogramDataset("processed/train")
dev_ds = MultiSpectrogramDataset("processed/dev")
eval_ds = MultiSpectrogramDataset("processed/eval")

# DataLoaders
train_loader = DataLoader(train_ds, batch_size=CNN_BATCH,
                           shuffle=True, num_workers=2, pin_memory=True)
dev_loader = DataLoader(dev_ds, batch_size=CNN_BATCH,
                        shuffle=False, num_workers=2, pin_memory=True)
eval_loader = DataLoader(eval_ds, batch_size=CNN_BATCH,
                          shuffle=False, num_workers=2, pin_memory=True)

print("\nDataLoaders ready ")
print(f"Train : {len(train_ds):,} segments")
print(f"Dev : {len(dev_ds):,} segments")
print(f"Eval : {len(eval_ds):,} segments")

# Extract unique file IDs from eval — needed for file-level aggregation later
eval_file_ids = np.array([
    "_".join(base.split("_seg")[0].split("_"))
    for base in eval_ds.base_names
])

seen = set()
unique_eval_file_ids = []
for fid in eval_file_ids:
    if fid not in seen:
        seen.add(fid)
        unique_eval_file_ids.append(fid)
print(f"Unique eval files : {len(unique_eval_file_ids):,}")

```

#Model Architecture Definitions

```

# — 1. CNN Branch (single spectrogram) —————
class CNNBranch(nn.Module):
    def __init__(self):

```

```

super().__init__()
self.features = nn.Sequential(
    # Block 1: (3,64,64) → (32,32,32)
    nn.Conv2d(3, 32, 3, padding=1),
    nn.BatchNorm2d(32), nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Dropout2d(0.2),      # ← dropout after block 1
    # Block 2: (32,32,32) → (64,16,16)
    nn.Conv2d(32, 64, 3, padding=1),
    nn.BatchNorm2d(64), nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Dropout2d(0.3),      # ← dropout after block 2
    # Block 3: (64,16,16) → (128,8,8)
    nn.Conv2d(64, 128, 3, padding=1),
    nn.BatchNorm2d(128), nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Dropout2d(0.4),      # ← dropout after block 3
    nn.Flatten()            # → 8192
)
def forward(self, x):
    return self.features(x)
# — 2. Multi-Branch CNN (3 spectrograms fused) —————
class MultiBranchCNN(nn.Module):
    BRANCH_DIM = 128 * 8 * 8 # 8192
    def __init__(self):
        super().__init__()
        self.stft_branch = CNNBranch()
        self.cqt_branch = CNNBranch()
        self.wav_branch = CNNBranch()
        self.classifier = nn.Sequential(
            nn.Linear(self.BRANCH_DIM * 3, 512), # 24576 → 512
            nn.ReLU(), nn.Dropout(0.6),
            nn.Linear(512, 256),

```



```

nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(256, 1)
)

```

```

def forward(self, inputs):
    stft, cqt, wav = inputs
    combined = torch.cat([
        self.stft_branch(stft),
        self.cqt_branch(cqt),
        self.wav_branch(wav)
    ], dim=1)
    return self.classifier(combined)

```

— 3. Multi-Branch ResNet-50 (pretrained) —————

```

class MultiBranchResNet(nn.Module):
    def __init__(self):
        super().__init__()
    def make_branch():
        m = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
        m.fc = nn.Identity()    # → 2048-dim output
        return m
    self.stft_branch = make_branch()
    self.cqt_branch = make_branch()
    self.wav_branch = make_branch()
    self.classifier = nn.Sequential(
        nn.Linear(2048 * 3, 512), # 6144 → 512
        nn.ReLU(), nn.Dropout(0.6),
        nn.Linear(512, 256),
        nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(256, 1)
    )
    def forward(self, inputs):
        stft, cqt, wav = inputs
        combined = torch.cat([

```

```

self.stft_branch(stft),
    self.cqt_branch(cqt),
    self.wav_branch(wav)
], dim=1)

return self.classifier(combined)

```

— 4. Whisper MLP —

```

class WhisperMLP_Fast(nn.Module):

```

```

    def __init__(self):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(), nn.Dropout(0.4),
            nn.Linear(256, 128),
            nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 1)
        )

```

```

    def forward(self, x):
        return self.mlp(x)

```

— Sanitycheck —

```

_i = (torch.randn(2, 3, 64, 64).to(device),) * 3
_cnn = MultiBranchCNN().to(device)
print(f'CNN output   : {_cnn(_i).shape}')
print(f'CNN params   : {sum(p.numel() for p in _cnn.parameters());}')
del _cnn

_res = MultiBranchResNet().to(device)
print(f'ResNet-50 output : {_res(_i).shape}')
print(f'ResNet-50 params : {sum(p.numel() for p in _res.parameters());}')
del _res, _i

_wh = WhisperMLP_Fast().to(device)
_wi = torch.randn(2, 512).to(device)
print(f'Whisper output: {_wh(_wi).shape}')
print(f'Whisper params: {sum(p.numel() for p in _wh.parameters());}')

```

```

del _wh, _wi
print("\nAll models defined ")
#Training loop and utility function definition
def train_spectrogram_model(model, train_loader, dev_loader,
                             epochs, lr, patience, save_path):
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
    best_dev_loss = float("inf")
    wait = 0
    history = {"train_loss": [], "dev_loss": [], "dev_acc": []}
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for (stft, cqt, wav), labels in train_loader:
            stft = stft.to(device)
            cqt = cqt.to(device)
            wav = wav.to(device)
            labels = labels.float().unsqueeze(1).to(device)
            optimizer.zero_grad()
            outputs = model((stft, cqt, wav))
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        model.eval()
        dev_loss, all_preds, all_labels_list = 0.0, [], []
        with torch.no_grad():
            for (stft, cqt, wav), labels in dev_loader:
                stft = stft.to(device)
                cqt = cqt.to(device)
                wav = wav.to(device)

```

```

labels = labels.float().unsqueeze(1).to(device)
outputs = model((stft, cqt, wav))
dev_loss += criterion(outputs, labels).item()
preds = (torch.sigmoid(outputs) > 0.5).int().cpu().numpy()
all_preds.extend(preds)
all_labels_list.extend(labels.int().cpu().numpy())
avg_train = running_loss / len(train_loader)
avg_dev = dev_loss / len(dev_loader)
acc = accuracy_score(all_labels_list, all_preds)
history["train_loss"].append(avg_train)
history["dev_loss"].append(avg_dev)
history["dev_acc"].append(acc)
scheduler.step()
print(f'Epoch [{epoch+1:02d}/{epochs}] '
      f'Train {avg_train:.4f} | '
      f'Dev {avg_dev:.4f} | '
      f'Acc {acc:.4f}', end='')
if avg_dev < best_dev_loss:
    best_dev_loss = avg_dev
    wait = 0
    torch.save(model.state_dict(), save_path)
    print(" saved")
else:
    wait += 1
    print(f' (patience {wait}/{patience})')
    if wait >= patience:
        print(" early stop")
        break
model.load_state_dict(torch.load(save_path, weights_only=True))
print(f'\nBest model loaded from {save_path}')
return history
def train_mlp(model, train_loader, dev_loader,

```

```

        epochs, lr, patience, save_path):
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
best_dev_loss = float("inf")
wait = 0
history = {"train_loss": [], "dev_loss": [], "dev_acc": []}
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for embeddings, labels in train_loader:
        embeddings = embeddings.to(device)
        labels = labels.float().unsqueeze(1).to(device)
        optimizer.zero_grad()
        outputs = model(embeddings)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    model.eval()
    dev_loss, all_preds, all_labels_list = 0.0, [], []
    with torch.no_grad():
        for embeddings, labels in dev_loader:
            embeddings = embeddings.to(device)
            labels = labels.float().unsqueeze(1).to(device)
            outputs = model(embeddings)
            dev_loss += criterion(outputs, labels).item()
            preds = (torch.sigmoid(outputs) > 0.5).int().cpu().numpy()
            all_preds.extend(preds)
            all_labels_list.extend(labels.int().cpu().numpy())
    avg_train = running_loss / len(train_loader)
    avg_dev = dev_loss / len(dev_loader)

```

```

acc      = accuracy_score(all_labels_list, all_preds)
history["train_loss"].append(avg_train)
history["dev_loss"].append(avg_dev)
history["dev_acc"].append(acc)
scheduler.step()
print(f'Epoch [{epoch+1:02d}/{epochs}] '
      f'Train {avg_train:.4f} | '
      f'Dev {avg_dev:.4f} | '
      f'Acc {acc:.4f}', end='')
if avg_dev < best_dev_loss:
    best_dev_loss = avg_dev
    wait = 0
    torch.save(model.state_dict(), save_path)
    print("  saved")
else:
    wait += 1
    print(f' (patience {wait}/{patience})')
    if wait >= patience:
        print("  early stop")
        break
model.load_state_dict(torch.load(save_path, weights_only=True))
print(f'\nBest model loaded from {save_path}')
return history

def get_spectrogram_probs(model, loader):
    model.eval()
    all_probs, all_labels = [], []
    with torch.no_grad():
        for (stft, cqt, wav), labels in loader:
            stft = stft.to(device)
            cqt = cqt.to(device)
            wav = wav.to(device)
            probs = torch.sigmoid(model((stft, cqt, wav))).cpu().numpy().flatten()

```

```

        all_probs.extend(probs)
        all_labels.extend(labels.numpy())
    return np.array(all_probs), np.array(all_labels)

def get_mlp_probs(model, loader):
    model.eval()
    all_probs, all_labels = [], []
    with torch.no_grad():
        for embeddings, labels in loader:
            embeddings = embeddings.to(device)
            probs = torch.sigmoid(model(embeddings)).cpu().numpy().flatten()
            all_probs.extend(probs)
            all_labels.extend(labels.numpy())
    return np.array(all_probs), np.array(all_labels)

def segment_to_file_level(probs, labels, segment_file_ids, unique_file_ids):
    file_probs, file_labels = [], []
    for file_id in unique_file_ids:
        indices = np.where(segment_file_ids == file_id)[0]
        file_probs.append(probs[indices].mean())
        file_labels.append(labels[indices[0]])
    return np.array(file_probs), np.array(file_labels)

def compute_eer(labels, probs):
    fpr, tpr, thresholds = roc_curve(labels, probs, pos_label=1)
    fnr = 1 - tpr
    eer_idx = np.argmin(np.abs(fpr - fnr))
    eer = (fpr[eer_idx] + fnr[eer_idx]) / 2
    return eer, thresholds[eer_idx]

def evaluate_all(probs, labels, name="Model"):
    preds = (probs > 0.5).astype(int)
    acc = accuracy_score(labels, preds)
    f1 = f1_score(labels, preds, zero_division=0)
    auc = roc_auc_score(labels, probs)
    eer, _ = compute_eer(labels, probs)

```

```

    print(f'{name:<35} Acc: {acc:.4f} F1: {f1:.4f} AUC: {auc:.4f} EER: {eer:.4f}')
    return {"accuracy": acc, "f1": f1, "auc": auc, "eer": eer}

print("All utilities defined ")

#Training the Multi-Branch CNN and evaluate

print("=" * 55)

print("Training Multi-Branch CNN")

print(f'Epochs: {CNN_EPOCHS} | LR: {CNN_LR} | Patience: {CNN_PATIENCE}')

print("=" * 55)

cnn_model = MultiBranchCNN().to(device)

cnn_history = train_spectrogram_model(
    cnn_model,
    train_loader,
    dev_loader,
    epochs = CNN_EPOCHS,
    lr = CNN_LR,
    patience = CNN_PATIENCE,
    save_path= "best_cnn.pth"
)

print("Evaluating CNN on eval set...")

cnn_seg_probs, cnn_seg_labels = get_spectrogram_probs(cnn_model, eval_loader)

cnn_file_probs, cnn_file_labels = segment_to_file_level(
    cnn_seg_probs,
    cnn_seg_labels,
    eval_file_ids,
    unique_eval_file_ids
)

print(f'Segments : {cnn_seg_probs.shape}')

print(f'Files : {cnn_file_probs.shape}')

print()

cnn_results = evaluate_all(cnn_file_probs, cnn_file_labels, "Multi-Branch CNN")

#Training the Multi-Branch ResNet-50 and evaluation

print("=" * 55)

```



```

print("Training Multi-Branch ResNet-50")

print(f'Epochs: {RESNET_EPOCHS} | LR: {RESNET_LR} | Patience:
{RESNET_PATIENCE}')

print("=" * 55)

resnet_model = MultiBranchResNet().to(device)

resnet_history = train_spectrogram_model(
    resnet_model,
    train_loader,
    dev_loader,
    epochs = RESNET_EPOCHS,
    lr = RESNET_LR,
    patience = RESNET_PATIENCE,
    save_path= "best_resnet.pth"
)

print("Evaluating ResNet-50 on eval set...")

resnet_seg_probs, resnet_seg_labels = get_spectrogram_probs(resnet_model, eval_loader)

resnet_file_probs, resnet_file_labels = segment_to_file_level(
    resnet_seg_probs,
    resnet_seg_labels,
    eval_file_ids,
    unique_eval_file_ids
)

print(f'Segments : {resnet_seg_probs.shape}')
print(f'Files : {resnet_file_probs.shape}')

print()

resnet_results = evaluate_all(resnet_file_probs, resnet_file_labels, "Multi-Branch ResNet-
50")

```

#Whisper Embedding and evaluation

Check available .npy files

```

for f in os.listdir("."):
    if f.endswith(".npy"):
        arr = np.load(f)
        print(f'{f} → {arr.shape}')

```

```

# Load embeddings
train_emb = torch.tensor(np.load("whisper_train_emb_final.npy"), dtype=torch.float32)
train_labels = torch.tensor(np.load("whisper_train_labels_final.npy"), dtype=torch.long)

dev_emb = torch.tensor(np.load("whisper_dev_emb_final.npy"), dtype=torch.float32)
dev_labels = torch.tensor(np.load("whisper_dev_labels.npy"), dtype=torch.long)

eval_emb = torch.tensor(np.load("whisper_matched_emb.npy"), dtype=torch.float32)
eval_labels_whisper = np.load("whisper_matched_labels.npy")

# DataLoaders
whisper_train_loader = DataLoader(
    TensorDataset(train_emb, train_labels),
    batch_size=WHISPER_BATCH, shuffle=True)
whisper_dev_loader = DataLoader(
    TensorDataset(dev_emb, dev_labels),
    batch_size=WHISPER_BATCH, shuffle=False)
whisper_eval_loader = DataLoader(
    TensorDataset(eval_emb, torch.tensor(eval_labels_whisper, dtype=torch.long)),
    batch_size=WHISPER_BATCH, shuffle=False)

print(f"Train : {train_emb.shape}")
print(f"Dev : {dev_emb.shape}")
print(f"Eval : {eval_emb.shape}")
print("Whisper loaders ready ")
print("=" * 55)
print("Training Whisper MLP")
print(f"Epochs: {WHISPER_EPOCHS} | LR: {WHISPER_LR} | Patience: {WHISPER_PATIENCE}")
print("=" * 55)
whisper_mlp = WhisperMLP_Fast().to(device)
whisper_history = train_mlp(
    whisper_mlp,
    whisper_train_loader,

```

```

    whisper_dev_loader,
    epochs = WHISPER_EPOCHS,
    lr = WHISPER_LR,
    patience = WHISPER_PATIENCE,
    save_path= "best_whisper.pth"
)
print("Evaluating Whisper MLP on eval set...")
whisper_probs_final, whisper_labels_final = get_mlp_probs(
    whisper_mlp, whisper_eval_loader
)
print(f"Files : {whisper_probs_final.shape}")
print()
whisper_results = evaluate_all(
    whisper_probs_final,
    whisper_labels_final,
    "Whisper MLP"
)

#Ensemble Fusion and Best Configuration Selection
print("=" * 65)
print("Individual Model Performance (File Level)")
print("=" * 65)
evaluate_all(cnn_file_probs,    cnn_file_labels,    "Multi-Branch CNN")
evaluate_all(resnet_file_probs, resnet_file_labels, "Multi-Branch ResNet-50")
evaluate_all(whisper_probs_final, whisper_labels_final, "Whisper MLP")
print("\n" + "=" * 65)
print("Ensemble Fusion Results")
print("=" * 65)
# Equal weight
ens_all = (cnn_file_probs + resnet_file_probs + whisper_probs_final) / 3
evaluate_all(ens_all, cnn_file_labels, "CNN + ResNet + Whisper (equal)")
# CNN + Whisper
ens_cw = (cnn_file_probs + whisper_probs_final) / 2

```

```

evaluate_all(ens_cw, cnn_file_labels, "CNN + Whisper")
# ResNet + Whisper
ens_rw = (resnet_file_probs + whisper_probs_final) / 2
evaluate_all(ens_rw, cnn_file_labels, "ResNet + Whisper")
# Whisper dominant
ens_w1 = (0.6 * whisper_probs_final + 0.3 * cnn_file_probs + 0.1 * resnet_file_probs)
evaluate_all(ens_w1, cnn_file_labels, "Weighted(W:0.6 C:0.3 R:0.1)")
# Whisper + CNN dominant
ens_w2 = (0.5 * whisper_probs_final + 0.4 * cnn_file_probs + 0.1 * resnet_file_probs)
evaluate_all(ens_w2, cnn_file_labels, "Weighted(W:0.5 C:0.4 R:0.1)")
# Whisper very dominant
ens_w3 = (0.7 * whisper_probs_final + 0.2 * cnn_file_probs + 0.1 * resnet_file_probs)
evaluate_all(ens_w3, cnn_file_labels, "Weighted(W:0.7 C:0.2 R:0.1)")
# Whisper only boosted by CNN
ens_w4 = (0.8 * whisper_probs_final + 0.2 * cnn_file_probs)
evaluate_all(ens_w4, cnn_file_labels, "Weighted(W:0.8 C:0.2)")
print("\n" + "=" * 65)
# Best by AUC
ensembles = {
    "CNN + ResNet + Whisper (equal)" : ens_all,
    "CNN + Whisper"                  : ens_cw,
    "ResNet + Whisper"                : ens_rw,
    "Weighted(W:0.6 C:0.3 R:0.1)"    : ens_w1,
    "Weighted(W:0.5 C:0.4 R:0.1)"    : ens_w2,
    "Weighted(W:0.7 C:0.2 R:0.1)"    : ens_w3,
    "Weighted(W:0.8 C:0.2)"          : ens_w4,
}
best_auc_name = max(ensembles, key=lambda k: roc_auc_score(cnn_file_labels,
ensembles[k]))
best_auc      = roc_auc_score(cnn_file_labels, ensembles[best_auc_name])
best_acc_name = max(ensembles, key=lambda k: accuracy_score(cnn_file_labels,
(ensembles[k] > 0.5).astype(int)))

```

```

best_acc    = accuracy_score(cnn_file_labels, (ensembles[best_acc_name] >
0.5).astype(int))

best_eer_name = min(ensembles, key=lambda k: compute_eer(cnn_file_labels,
ensembles[k])[0])

best_eer    = compute_eer(cnn_file_labels, ensembles[best_eer_name])[0]

print(f"\n Best AUC : {best_auc_name} → {best_auc:.4f}")
print(f" Best Acc : {best_acc_name} → {best_acc:.4f}")
print(f" Best EER : {best_eer_name} → {best_eer:.4f}")

#Comprehensive Visualization

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# — Ensemble —————
ens_all = (cnn_file_probs + resnet_file_probs + whisper_probs_final) / 3
ens_results = evaluate_all(ens_all, cnn_file_labels, "CNN + ResNet-50 + Whisper")

# — Data prep —————
models = {
    "CNN"          : (cnn_file_probs,   cnn_file_labels),
    "ResNet-50"     : (resnet_file_probs, resnet_file_labels),
    "Whisper"       : (whisper_probs_final, whisper_labels_final),
    "CNN+ResNet50+Whisper" : (ens_all,      cnn_file_labels),
}

names  = list(models.keys())
accs   = [accuracy_score(l, (p>0.5).astype(int)) for p,l in models.values()]
f1s    = [f1_score(l, (p>0.5).astype(int), zero_division=0) for p,l in models.values()]
aucs   = [roc_auc_score(l, p) for p,l in models.values()]
eers   = [compute_eer(l, p)[0] for p,l in models.values()]

colors = ["#4C72B0", "#DD8452", "#55A868", "#C44E52"]
ens_color = "#C44E52"

fig = plt.figure(figsize=(18, 14))

fig.suptitle("ASVspoof2019 — Model Comparison & Ensemble", fontsize=16,
fontweight="bold", y=0.98)

```

```

gs = gridspec.GridSpec(2, 3, figure=fig, hspace=0.4, wspace=0.35)

# — 1. Accuracy bar —————
ax1 = fig.add_subplot(gs[0, 0])
bars = ax1.bar(names, accs, color=colors, edgecolor="white", linewidth=0.8)
ax1.axhline(0.87, color="gray", linestyle="--", linewidth=1, label="87% target")
ax1.set_title("Accuracy", fontweight="bold")
ax1.set_ylim(0.75, 1.01)
ax1.set_ylabel("Accuracy")
ax1.tick_params(axis="x", rotation=15)
ax1.legend(fontsize=8)
for bar, val in zip(bars, accs):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.003,
             f'{val:.4f}', ha="center", va="bottom", fontsize=8, fontweight="bold")

# — 2. AUC bar —————
ax2 = fig.add_subplot(gs[0, 1])
bars2 = ax2.bar(names, aucs, color=colors, edgecolor="white", linewidth=0.8)
ax2.set_title("AUC", fontweight="bold")
ax2.set_ylim(0.75, 1.01)
ax2.set_ylabel("AUC")
ax2.tick_params(axis="x", rotation=15)
for bar, val in zip(bars2, aucs):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.003,
             f'{val:.4f}', ha="center", va="bottom", fontsize=8, fontweight="bold")

# — 3. EER bar —————
ax3 = fig.add_subplot(gs[0, 2])
bars3 = ax3.bar(names, eers, color=colors, edgecolor="white", linewidth=0.8)
ax3.set_title("EER (lower = better)", fontweight="bold")
ax3.set_ylabel("EER")
ax3.tick_params(axis="x", rotation=15)
for bar, val in zip(bars3, eers):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
             f'{val:.4f}', ha="center", va="bottom", fontsize=8, fontweight="bold")

```

```

# — 4. ROC curves —————
ax4 = fig.add_subplot(gs[1, 0])
for (name, (probs, labels)), color in zip(models.items(), colors):
    fpr, tpr, _ = roc_curve(labels, probs)
    auc_val = roc_auc_score(labels, probs)
    lw = 2.5 if name == "CNN+ResNet50+Whisper" else 1.2
    ax4.plot(fpr, tpr, color=color, lw=lw, label=f'{name} (AUC={auc_val:.3f})')
ax4.plot([0,1],[0,1], "k--", lw=0.8)
ax4.set_title("ROC Curves", fontweight="bold")
ax4.set_xlabel("False Positive Rate")
ax4.set_ylabel("True Positive Rate")
ax4.legend(fontsize=7)

# — 5. Training loss curves —————
ax5 = fig.add_subplot(gs[1, 1])
ax5.plot(cnn_history["train_loss"], color="#4C72B0", lw=1.5, label="CNN Train")
ax5.plot(cnn_history["dev_loss"], color="#4C72B0", lw=1.5, linestyle="--", label="CNN Dev")
ax5.plot(resnet_history["train_loss"], color="#DD8452", lw=1.5, label="ResNet Train")
ax5.plot(resnet_history["dev_loss"], color="#DD8452", lw=1.5, linestyle="--", label="ResNet Dev")
ax5.plot(whisper_history["train_loss"], color="#55A868", lw=1.5, label="Whisper Train")
ax5.plot(whisper_history["dev_loss"], color="#55A868", lw=1.5, linestyle="--", label="Whisper Dev")
ax5.set_title("Training vs Dev Loss", fontweight="bold")
ax5.set_xlabel("Epoch")
ax5.set_ylabel("Loss")
ax5.legend(fontsize=7)

# — 6. Confusion matrix — ensemble only —————
ax6 = fig.add_subplot(gs[1, 2])
ens_preds = (ens_all > 0.5).astype(int)
cm = confusion_matrix(cnn_file_labels, ens_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Spoof", "Bonafide"])

```

```

disp.plot(ax=ax6, colorbar=False, cmap="Blues")
ax6.set_title("Ensemble Confusion Matrix", fontweight="bold")
plt.savefig("ensemble_results.png", dpi=150, bbox_inches="tight")
plt.show()
print("Plot saved as ensemble_results.png ")

#saving
import pickle
from datetime import datetime
save_dir = "saved_models"
os.makedirs(save_dir, exist_ok=True)

# — Model weights —————
torch.save(cnn_model.state_dict(), f'{save_dir}/best_cnn.pth')
torch.save(resnet_model.state_dict(), f'{save_dir}/best_resnet50.pth')
torch.save(whisper_mlp.state_dict(), f'{save_dir}/best_whisper_mlp.pth')
print(" Model weights saved")

# — Probabilities —————
np.save(f'{save_dir}/cnn_file_probs.npy",  cnn_file_probs)
np.save(f'{save_dir}/cnn_file_labels.npy",  cnn_file_labels)
np.save(f'{save_dir}/resnet_file_probs.npy",  resnet_file_probs)
np.save(f'{save_dir}/resnet_file_labels.npy",  resnet_file_labels)
np.save(f'{save_dir}/whisper_probs.npy",  whisper_probs_final)
np.save(f'{save_dir}/whisper_labels.npy",  whisper_labels_final)
np.save(f'{save_dir}/ensemble_probs.npy",  ens_all)
print(" Probabilities saved")

# — Training histories —————
with open(f'{save_dir}/histories.pkl", "wb") as f:
    pickle.dump({
        "cnn" : cnn_history,
        "resnet" : resnet_history,
        "whisper": whisper_history
    }, f)

```



```

print(" Training histories saved")

# — Results summary —————
summary = {
    "timestamp" : datetime.now().strftime("%Y-%m-%d %H:%M"),
    "CNN"       : cnn_results,
    "ResNet50"  : resnet_results,
    "Whisper"   : whisper_results,
    "Ensemble"  : ens_results,
}

with open(f'{save_dir}/results_summary.pkl', "wb") as f:
    pickle.dump(summary, f)

print(" Results summary saved")

# — Visualization —————
import shutil

shutil.copy("ensemble_results.png", f'{save_dir}/ensemble_results.png')

print(" Visualization saved")

# — Print summary —————
print("\n" + "=" * 50)

print("FINAL RESULTS SUMMARY")

print("=" * 50)

print(f'{"Model":<30} {"Acc":>8} {"AUC":>8} {"EER":>8}')
```

```

print("-" * 50)

print(f'{"CNN":<30} {cnn_results["accuracy"]:>8.4f} {cnn_results["auc"]:>8.4f}
{cnn_results["eer"]:>8.4f}')
```

```

print(f'{"ResNet-50":<30} {resnet_results["accuracy"]:>8.4f} {resnet_results["auc"]:>8.4f}
{resnet_results["eer"]:>8.4f}')
```

```

print(f'{"Whisper MLP":<30} {whisper_results["accuracy"]:>8.4f}
{whisper_results["auc"]:>8.4f} {whisper_results["eer"]:>8.4f}')
```

```

print(f'{"CNN + ResNet50 + Whisper":<30} {ens_results["accuracy"]:>8.4f}
{ens_results["auc"]:>8.4f} {ens_results["eer"]:>8.4f}')
```

```

print("=" * 50)

print(f'\nAll files saved to: {save_dir}/')

print(os.listdir(save_dir))

```

#classification report

— Classification Report — All Models —

```
import numpy as np
```

```
from sklearn.metrics import classification_report
```

```
cnn_probs = np.load('saved_models/cnn_file_probs.npy')
```

```
cnn_labels = np.load('saved_models/cnn_file_labels.npy')
```

```
resnet_probs = np.load('saved_models/resnet_file_probs.npy')
```

```
whisper_probs = np.load('saved_models/whisper_probs.npy')
```

```
whisper_lbls = np.load('saved_models/whisper_labels.npy')
```

```
ens_probs = np.load('saved_models/ensemble_probs.npy')
```

```
cnn_preds = (cnn_probs > 0.5).astype(int)
```

```
resnet_preds = (resnet_probs > 0.5).astype(int)
```

```
whisper_preds = (whisper_probs > 0.5).astype(int)
```

```
ens_preds = (ens_probs > 0.5).astype(int)
```

```
models = {
```

```
    'Multi-Branch CNN' : (cnn_labels, cnn_preds),
```

```
    'Multi-Branch ResNet-50' : (cnn_labels, resnet_preds),
```

```
    'Whisper MLP' : (whisper_lbls, whisper_preds),
```

```
    'Ensemble (CNN+ResNet+Whisper)' : (cnn_labels, ens_preds),
```

```
}
```

```
for name, (labels, preds) in models.items():
```

```
    print('='*55)
```

```
    print(f' {name}')
```

```
    print('='*55)
```

```
    print(classification_report(labels, preds,
```

```
                                target_names=['Spoof','Bonafide'],
```

```
                                digits=4))
```

#widget for check on unseen data

— paste this into its OWN separate cell and run it alone —

```
uploader = widgets.FileUpload(
    accept='.wav,.flac,.mp3,.ogg,.m4a',
    multiple=False,
    description='Upload Audio',
    button_style='primary',
    layout=widgets.Layout(width='200px')
)

run_btn = widgets.Button(
    description='▶ Run Inference',
    button_style='success',
    layout=widgets.Layout(width='180px')
)

out = widgets.Output()

def on_run(_):
    with out:
        clear_output(wait=True)
        if not uploader.value:
            print('⚠ Please upload an audio file first.')
            return

        val = uploader.value

        if isinstance(val, tuple):
            item = val[0]
        elif isinstance(val, dict):
```

```

        item = list(val.values())[0]
    else:
        item = val

    if isinstance(item, dict):
        fname = item.get('name') or item.get('filename') or item.get('metadata',
{}).get('name', 'audio')
        content = item.get('content') or item.get('data') or b''
    else:
        fname = getattr(item, 'name', None) or getattr(item, 'filename', 'audio')
        content = getattr(item, 'content', None) or getattr(item, 'data', b'')

    print(f📁 Detected file: {fname} | size: {len(bytes(content)):.} bytes')
    predict_and_plot(bytes(content), fname)

run_btn.on_click(on_run)

display(widgets.VBox([
    widgets.HTML('<h3> 🎧 Unseen Audio Tester</h3>'),
    widgets.HBox([uploader, run_btn]),
    out
]))

```